Pinboard Notes

Complete Architectural Specification & Unabridged Source Code

SOURCE FILES	TOTAL CODE	FEATURE SPECS	RUNTIME
23 Files	**5,325 Lines**	**10 Core Modules**	**React 19 + Vite**

Executive Summary

Pinboard Notes is a production-grade hierarchical note-taking application engineered with React 19, TypeScript, and Tailwind CSS. This comprehensive documentation captures every feature down to the last bit: hierarchical tree persistence, chronological pinned note importance, custom hold-and-drag note arrangement, editable detached bubble pins with transparency and viewport cropping, standout trigger controls, smart Recycle Bin restoration to parent writing pages, full-text instant search, and dual PDF generation engines.

Document Structure & Contents

PART I: ARCHITECTURAL SPECIFICATIONS & FEATURE MANUAL — **Page 2 - 4**

- **[F01]** Hierarchical Tree Architecture & Path Navigation
- **[F02]** Hierarchy of Importance: Pinning & Chronological Sorting
- **[F03]** Hold & Arrange Notes' Order (Manual Drag & Drop)
- **[F04]** Editable Floating Bubble Pin & Customizable Viewport
- **[F05]** Unique Standout Bubble Pop Button
- **[F06]** Recycle Bin & Pinned Note Restoration Flow
- **[F07]** Archive System with Instant Restoration
- **[F08]** Instant Global Full-Text Search
- **[F09]** Dual PDF Export Engines (Note & Full Codebase)
- **[F10]** Design System & Responsive Dual Themes

PART II: UNABRIDGED SOURCE CODE (23 FILES) — **Page 5+**

`metadata.json`	7 lines	Configuration
`package.json`	41 lines	Configuration
`vite.config.ts`	24 lines	Build Setup
`tsconfig.json`	27 lines	Configuration
`index.html`	17 lines	Web Entry
`src/main.tsx`	11 lines	Web Entry
`src/types.ts`	44 lines	Data Models
`src/vite-env.d.ts`	7 lines	Types
`src/App.tsx`	509 lines	Application Core
`src/index.css`	1607 lines	Design System

... and 13 additional source and configuration files printed completely below.

PART I: ARCHITECTURAL SPECIFICATIONS & FEATURE MANUAL

F01 — Hierarchical Tree Architecture & Path Navigation

Multi-level nested note trees with immutable updates and breadcrumb navigation

• Data Model: Every note is represented as a recursive NoteNode containing id, title, content (HTML string), archived (boolean), pinned (boolean), pinnedAt (timestamp | null), order (number), and children (NoteNode[]).

• Navigation Stack: Navigation state is maintained via a string array stack: string[] representing the active path from "root" to the current node. Breadcrumbs display the ancestral trail.

• Immutable Mutations: All tree updates use deep cloning (cloneTree) ensuring pure React state transitions, zero side-effects, and reliable re-rendering.

• Auto-Persistence: State modifications are immediately synced to localStorage; optional host-storage writes are serialized to protect recent edits.

F02 — Hierarchy of Importance: Pinning & Chronological Sorting

Pinned notes elevate with structural priority and chronological ordering

• Significance Over Glow: Pinned notes are not merely highlighted with a visual glow; they are given explicit organizational hierarchy at the top of the container.

• Chronological Timestamping: When a note is pinned, node.pinnedAt is set to Date.now(). Pinned notes are dynamically sorted descending by pinnedAt, so the most recently pinned note always appears first.

• Clear Visual Separation: BoardView splits items into two distinct sections: "Pinned Notes (Hierarchy of Importance)" at the top with a distinct pin badge, and "Notes" below.

• Graceful Unpinning: When unpinned, node.pinned is set to false and node.pinnedAt is reset to null. The note returns back to its standard order in the regular notes section without data loss.

F03 — Hold & Arrange Notes' Order (Manual Drag & Drop)

Touch & pointer-driven hold-and-drag reordering for custom prioritization

• Grip Handles & Pointer Events: Each NoteCard features a dedicated drag grip handle (GripVertical) and handles onPointerDown/onDragStart events.

• Drop Indicator & Animation: While dragging, cards calculate drag index and render visual insertion guides with drop target styling.

• Persistent Reorder Algorithm: The reorderChildren utility mutates the order property of child nodes and updates the parent tree in state, ensuring customized ordering persists across reloads.

• Scoped Dragging: Reordering works independently within both the pinned section and the unpinned section, giving users full layout autonomy.

F04 — Editable Floating Bubble Pin & Customizable Viewport

Detached, floating, resizable and croppable note workspace

• Detached Multitasking: Any note can be popped out into a floating bubble pin that hovers above the entire application.

• Full Bi-directional Editing: The bubble pin is completely editable with a rich text area. Typing inside the bubble immediately mutates the underlying tree node via onUpdateNodeContent.

• Comprehensive Settings Panel: A toggleable settings modal on the bubble pin provides dynamic sliders for:

• • Transparency Slider: Adjusts container opacity from 20% (see-through reference) to 100% (solid focus).

• • Bubble Pop Size: Adjusts the collapsed/mini badge trigger diameter from 48px to 96px.

• • Expansion Size: Controls expanded panel width (320px to 720px) and height (280px to 620px).

• • Viewport Cropping Engine: When enabled, users can crop top and bottom insets to restrict the visible window exclusively to a specific line, code snippet, or paragraph.

• Draggable Physics: Users can drag the bubble pin anywhere across the viewport via pointer drag on the header handle.

PART I: ARCHITECTURAL SPECIFICATIONS (CONTINUED)

F05 — Unique Standout Bubble Pop Button

Distinctive visual design and identity for the bubble pop feature

- Distinct Identity: The Bubble Pop trigger is visually differentiated from the standard Pin+ button.
- Vibrant Gradient & Halo: Styled with a radiant linear gradient (linear-gradient(135deg, #FF6B6B, #9B51E0)), subtle ambient drop-shadow, and pulsing halo.
- Animated Sparkle Icon: Houses an animated Sparkles icon with gentle rotation and a clear "Bubble Pin" text label in writing mode.
- Instant Elevation: One click transforms any active note into an editable detached floating bubble pin.

F06 — Recycle Bin & Pinned Note Restoration Flow

Safe deletion, parent tracking, and smart contextual restoration

- Parent Lineage Preservation: When a note is deleted, it is placed in the trash array as a TrashEntry with its parentId, deletedAt timestamp, and deep-cloned node data.
- Re-embedding Content Chips: Upon restoration, the note is re-inserted into parent.children. If parent.content lacks the interactive chip, makeChipHtml re-embeds the note chip into the parent note content.
- Parent Page Landing Logic: Restoring a pinned note correctly routes the user when clicking the parent page to the parent pinned note's writing page (where the restored note is pinned as a chip), rather than entering directly into the restored note.
- Permanent Deletion: Items in the Recycle Bin can be selectively restored or permanently expunged.

F07 — Archive System with Instant Restoration

Declutter boards by archiving completed or reference notes

- Soft Archival: Setting node.archived = true hides notes from the active board without altering their internal hierarchy.
- Dedicated Archive View: Users can view all archived notes for the current page container, complete with date tags and pin badges.
- One-Click Restoration: Unarchiving seamlessly restores the note to the active board and ensures chips in parent content are synced.

F08 — Instant Global Full-Text Search

Comprehensive search across titles and rich text with highlighted matches

- Full Tree Traversal: The search engine recursively scans every node in the tree hierarchy.
- HTML Tag Stripping: Content is stripped of HTML tags before querying, ensuring clean, accurate text comparisons.
- Snippet Context Generation: Matches generate contextual text excerpts with the exact search query emphasized in highlighted badges.
- Deep Stack Reconstruction: Clicking any search result reconstructs the complete parent path array (stack: result.path) and transitions immediately to writing mode.

F09 — Dual PDF Export Engines (Note & Full Codebase)

High-fidelity document generation powered by jsPDF

- Note-Level PDF Export: Users can export any note into a clean, formatted A4 PDF containing its title, metadata, and stripped body text.
- Complete Codebase PDF Generator: Bundles every source file, configuration file, build script, and architectural documentation into a single comprehensive PDF.
- Table of Contents & Line Numbering: Code files are rendered with monospace typography, line numbering, file metadata badges, and running headers/footers.
- Static & Dynamic Availability: Pre-generated to public/Pinboard_Notes_Codebase.pdf and dynamically downloadable through the in-app modal.

PART I: ARCHITECTURAL SPECIFICATIONS (CONTINUED)

F10 — Design System & Responsive Dual Themes

Midnight Dark and Warm Parchment Light modes with CSS variable tokens

- Midnight Dark Theme: High contrast deep blue palette (--bg1: #0A1024, --bg2: #131B3B, --accent1: #3A5FE0, --pin-active: #FFC966).
- Warm Parchment Light Theme: Paper-inspired aesthetic (--bg1: #F3EAD9, --bg2: #F6EFE1, --accent1: #E8823D, --pin-active: #C9631F).
- Zero Layout Shift: All UI controls, modals, sheets, and floating toolbars dynamically adapt to theme tokens with seamless CSS transitions.
- Mobile & Desktop Responsive: Optimized for phone screens, tablets, and wide desktop displays with responsive split layouts.

metadata.json

Configuration | 7 Lines

Role: Application metadata, identity, and studio capabilities

```
1 |   {
2 |     "name": "Pinboard Notes",
3 |     "description": "A feature-packed hierarchical note-taking app with bubble pins, custom drag-ordering, and
  > | editable floating panels.",
4 |     "requestFramePermissions": [],
5 |     "majorCapabilities": ["MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"]
6 |   }
7 |
```

package.json

Configuration | 41 Lines

Role: NPM package dependencies, scripts, and build tasks

```
{
  "name": "react-example",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --port=3000 --host=0.0.0.0",
    "generate:pdf": "tsx scripts/generate_code_pdf.ts",
    "generate:zip": "python3 scripts/create_project_zip.py",
    "prebuild": "tsx scripts/generate_code_pdf.ts && python3 scripts/create_project_zip.py",
    "build": "vite build",
    "preview": "vite preview",
    "clean": "rm -rf dist server.js",
    "lint": "tsc --noEmit"
  },
  "dependencies": {
    "@google/genai": "^2.4.0",
    "@tailwindcss/vite": "^4.1.14",
    "@vitejs/plugin-react": "^5.0.4",
    "dotenv": "^17.2.3",
    "express": "^4.21.2",
    "jspdf": "^4.2.1",
    "jszip": "^3.10.2",
    "lucide-react": "^0.546.0",
    "motion": "^12.23.24",
    "react": "^19.0.1",
    "react-dom": "^19.0.1",
    "vite": "^6.2.3"
  },
  "devDependencies": {
    "@types/node": "^22.14.0",
    "autoprefixer": "^10.4.21",
    "esbuild": "^0.25.0",
    "tailwindcss": "^4.1.14",
    "tsx": "^4.21.0",
    "typescript": "~5.8.2",
    "vite": "^6.2.3",
    "@types/express": "^4.17.21"
  }
}

```

vite.config.ts

Build Setup | 24 Lines

Role: Vite build tool configuration and Tailwind CSS integration

```
1 |   import tailwindcss from '@tailwindcss/vite';
2 |   import react from '@vitejs/plugin-react';
3 |   import path from 'path';
4 |   import {defineConfig} from 'vite';
5 |
6 |   export default defineConfig(() => {
7 |     return {
8 |       base: './',
9 |       plugins: [react(), tailwindcss()],
10 |       resolve: {
11 |         alias: {
12 |           '@': path.resolve(__dirname, '.'),
13 |         },
14 |       },
15 |       server: {
16 |         // HMR is disabled in AI Studio via DISABLE_HMR env var.
17 |         // Do not modifyâ€”file watching is disabled to prevent flickering during agent edits.
18 |         hmr: process.env.DISABLE_HMR !== 'true',
19 |         // Disable file watching when DISABLE_HMR is true to save CPU during agent edits.
20 |         watch: process.env.DISABLE_HMR === 'true' ? null : {},
21 |       },
22 |     };
23 |   });
24 |
```

tsconfig.json

Configuration | 27 Lines

Role: TypeScript compiler configuration and strict path aliases

```
{
  "compilerOptions": {
    "target": "ES2022",
    "experimentalDecorators": true,
    "useDefineForClassFields": false,
    "module": "ESNext",
    "lib": [
      "ES2022",
      "DOM",
      "DOM.Iterable"
    ],
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "isolatedModules": true,
    "moduleDetection": "force",
    "allowJs": true,
    "jsx": "react-jsx",
    "paths": {
      "@/*": [
        "./*"
      ]
    },
    "allowImportingTsExtensions": true,
    "noEmit": true
  }
}

```

index.html

Web Entry | 17 Lines

Role: HTML entry point with viewport, title, and OpenGraph tags

```
1 |   <!doctype html>
2 |   <html lang="en">
3 |     <head>
4 |       <meta charset="UTF-8" />
5 |       <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 |       <title>Pinboard Notes</title>
7 |       <meta name="description" content="A feature-packed hierarchical note-taking app with bubble pins, custom
  > | drag-ordering, and editable floating panels." />
8 |       <meta property="og:title" content="Pinboard Notes" />
9 |       <meta property="og:description" content="A feature-packed hierarchical note-taking app with bubble pins,
  > | custom drag-ordering, and editable floating panels." />
10 |     </head>
11 |     <body>
12 |       <div id="root"></div>
13 |       <script type="module" src="/src/main.tsx"></script>
14 |     </body>
15 |   </html>
16 |
17 |
```

src/main.tsx

Web Entry | 11 Lines

Role: React 19 entry point and root DOM container mounting

```
1 |   import {StrictMode} from 'react';
2 |   import {createRoot} from 'react-dom/client';
3 |   import App from './App.tsx';
4 |   import './index.css';
5 |
6 |   createRoot(document.getElementById('root')!).render(
7 |     <StrictMode>
8 |       <App />
9 |     </StrictMode>,
10 |   );
11 |
```

src/types.ts

Data Models | 44 Lines

Role: Type definitions for NoteNode, TrashEntry, and BubbleSettings

```
export interface NoteNode {
  id: string;
  title: string;
  content: string;
  archived: boolean;
  pinned: boolean;
  pinnedAt?: number | null; // Timestamp when note was pinned (for hierarchy ordering)
  order?: number;           // Manual sorting index within its parent container
  children: NoteNode[];
  createdAt?: number;
  updatedAt?: number;
}

export interface TrashEntry {
  id: string;
  node: NoteNode;
  parentId: string;
  deletedAt: number;
}

export interface BubbleSettings {
  transparency: number;     // 0.2 to 1.0 (opacity)
  bubblePopSize: number;    // Diameter in px (48 to 96)
  expansionWidth: number;   // In px (320 to 720) or percentage
  expansionHeight: number;  // In px or vh (280 to 600)
  cropEnabled: boolean;     // Whether viewport cropping is active
  cropTop: number;          // Crop inset top in px
  cropBottom: number;       // Crop inset bottom in px
  cropHeight: number;       // Cropped visible window height
}

export type ViewMode = 'board' | 'write' | 'archive' | 'trash';
export type ThemeMode = 'dark' | 'light';

export interface SearchResult {
  id: string;
  title: string;
  path: string[];
  archived: boolean;
  pinned: boolean;
  matchType: 'title' | 'content';
  snippet: string;
}

```

src/vite-env.d.ts

Types | 7 Lines

Role: Vite environment and module declaration for raw imports

```
1 |   /// <reference types="vite/client" />
2 |
3 |   declare module '*?raw' {
4 |     const content: string;
5 |     export default content;
6 |   }
7 |
```

src/App.tsx

Application Core | 509 Lines

Role: Main application controller, state machine, and PDF integration

```tsx
import React, { useState, useEffect, useLayoutEffect, useRef, useCallback } from 'react';
import { jsPDF } from 'jspdf';
import { NoteNode, TrashEntry, BubbleSettings, ViewMode, ThemeMode } from './types';
import { loadAppData, saveAppData } from './utils/storage';
import {
  createRootNode,
  cloneTree,
  findNode,
  findParentNode,
  findPathIds,
  deleteNodeById,
  makeChipHtml,
  contentHasChip,
  generateUid,
  stripHtml,
} from './utils/tree';
import { TopBar } from './components/TopBar';
import { BoardView } from './components/BoardView';
import { EditorView } from './components/EditorView';
import { BubblePin, DEFAULT_BUBBLE_SETTINGS } from './components/BubblePin';
import { ActionSheet } from './components/ActionSheet';
import { SearchModal } from './components/SearchModal';
import { ArchiveTrashView } from './components/ArchiveTrashView';
import { CodeExportModal } from './components/CodeExportModal';

export default function App() {
  const [tree, setTree] = useState<NoteNode | null>(null);
  const [trash, setTrash] = useState<TrashEntry[]>([]);
  const [theme, setTheme] = useState<ThemeMode>('dark');
  const [bubbleSettings, setBubbleSettings] = useState<BubbleSettings>(DEFAULT_BUBBLE_SETTINGS);
  const [loaded, setLoaded] = useState(false);
  const [loadError, setLoadError] = useState(false);
  const [saveError, setSaveError] = useState(false);

  const [stack, setStack] = useState<string[]>(['root']);
  const [mode, setMode] = useState<ViewMode>('board');

  const [longPressId, setLongPressId] = useState<string | null>(null);
  const [floatingId, setFloatingId] = useState<string | null>(null);

  const [toast, setToast] = useState<string | null>(null);
  const [searchOpen, setSearchOpen] = useState(false);
  const [codeExportOpen, setCodeExportOpen] = useState(false);
  const [trashAreaId, setTrashAreaId] = useState<string>('root');

  const toastTimeout = useRef<any>(null);

  // Load initial app data
  useEffect(() => {
    let cancelled = false;
    (async () => {
      try {
        const data = await loadAppData();
        if (cancelled) return;
        if (data) {
          setTree(data.tree || createRootNode());
          setTrash(data.trash || []);
          setTheme(data.theme || 'dark');
          if (data.bubbleSettings) {
            setBubbleSettings({ ...DEFAULT_BUBBLE_SETTINGS, ...data.bubbleSettings });
          }
        } else {
          setTree(createRootNode());
        }
      } catch (err) {
```

src/App.tsx (continued from previous page)

```
          if (!cancelled) setLoadError(true);
        } finally {
          if (!cancelled) setLoaded(true);
        }
      })();
      return () => { cancelled = true; };
    }, []);

    // Sync theme to root element
    useEffect(() => {
      document.documentElement.setAttribute('data-theme', theme);
    }, [theme]);

    // Save changes to storage
    useLayoutEffect(() => {
      if (!loaded || !tree || loadError) return;
      // localStorage is written synchronously before the optional remote save.
      void saveAppData({ tree, trash, theme, bubbleSettings }).then((saved) => {
        setSaveError(!saved);
      });
    }, [tree, trash, theme, bubbleSettings, loaded, loadError]);

    useEffect(() => {
      return () => clearTimeout(toastTimeout.current);
    }, []);

    const showToast = useCallback((msg: string) => {
      setToast(msg);
      clearTimeout(toastTimeout.current);
      toastTimeout.current = setTimeout(() => setToast(null), 1800);
    }, []);

    const updateTree = useCallback((mutator: (tree: NoteNode) => void) => {
      setTree((prev) => {
        if (!prev) return createRootNode();
        const next = cloneTree(prev);
        mutator(next);
        return next;
      });
    }, []);

    const currentId = stack[stack.length - 1] || 'root';
    const currentNode = tree ? findNode(tree, currentId) || tree : null;
    const longPressNode = tree && longPressId ? findNode(tree, longPressId) : null;
    const floatingNode = tree && floatingId ? findNode(tree, floatingId) : null;
    const trashAreaNode = tree ? findNode(tree, trashAreaId) : null;

    // Add new note
    const handleAddChild = () => {
      const newId = generateUid();
      updateTree((t) => {
        const parent = findNode(t, currentId) || t;
        if (!parent.children) parent.children = [];
        parent.children.push({
          id: newId,
          title: 'Untitled Note',
          content: '',
          archived: false,
          pinned: false,
          pinnedAt: null,
          order: parent.children.length,
          children: [],
          createdAt: Date.now(),
          updatedAt: Date.now(),
        });
      });
      showToast('New note created');
    };

```

src/App.tsx (continued from previous page)

```
135 |     // Insert a nested pinned note chip into the editor writing page
136 |     const handleInsertNestedNote = () => {
137 |       const newId = generateUid();
138 |       const title = 'Untitled Note';
139 |       updateTree((t) => {
140 |         const parent = findNode(t, currentId) || t;
141 |         if (!parent.children) parent.children = [];
142 |         parent.children.push({
143 |           id: newId,
144 |           title,
145 |           content: '',
146 |           archived: false,
147 |           pinned: false,
148 |           pinnedAt: null,
149 |           order: parent.children.length,
150 |           children: [],
151 |           createdAt: Date.now(),
152 |           updatedAt: Date.now(),
153 |         });
154 |         parent.content = (parent.content || '') + makeChipHtml(newId, title, false) + '<br>';
155 |         parent.updatedAt = Date.now();
156 |       });
157 |       showToast('Nested note inserted');
158 |     };
159 |
160 |     const handleOpenNote = (id: string) => {
161 |       const path = tree && findPathIds(tree, id);
162 |       if (!path) return;
163 |       setMode('write');
164 |       setStack(path);
165 |     };
166 |
167 |     const handleBack = () => {
168 |       if (stack.length <= 1) return;
169 |       const next = stack.slice(0, -1);
170 |       setStack(next);
171 |       if (next[next.length - 1] === 'root') setMode('board');
172 |     };
173 |
174 |     // Toggle Pin Note with hierarchy of importance timestamp
175 |     const handleTogglePin = (id: string) => {
176 |       const nowPinned = !tree || !findNode(tree, id)?.pinned;
177 |       updateTree((t) => {
178 |         const node = findNode(t, id);
179 |         if (node) {
180 |           node.pinned = !node.pinned;
181 |           node.pinnedAt = node.pinned ? Date.now() : null;
182 |         }
183 |       });
184 |       setLongPressId(null);
185 |       showToast(nowPinned ? 'Pinned to top of page' : 'Unpinned — returned to normal list');
186 |     };
187 |
188 |     // Copy note content
189 |     const handleCopyNote = async (node: NoteNode) => {
190 |       const text = `${node.title || 'Untitled Note'}\n\n${stripHtml(node.content)}`;
191 |       let success = false;
192 |       try {
193 |         if (navigator.clipboard && navigator.clipboard.writeText) {
194 |           await navigator.clipboard.writeText(text);
195 |           success = true;
196 |         }
197 |       } catch (e) {
198 |         success = false;
199 |       }
200 |       if (!success) {
201 |         const ta = document.createElement('textarea');
202 |         ta.value = text;
203 |         ta.style.position = 'fixed';
```

src/App.tsx (continued from previous page)

```
        ta.style.opacity = '0';
        document.body.appendChild(ta);
        ta.focus();
        ta.select();
        try {
          success = document.execCommand('copy');
        } catch (e) {
          success = false;
        }
        document.body.removeChild(ta);
      }
      showToast(success ? 'Copied note content' : 'Could not copy note');
      setLongPressId(null);
    };

    // Download Note PDF helper
    const handleDownloadPdf = async (node: NoteNode) => {
      showToast('Generating PDF...');
      try {
        const doc = new jsPDF({ unit: 'pt', format: 'a4' });
        const marginX = 48;
        const pageWidth = doc.internal.pageSize.getWidth();
        const pageHeight = doc.internal.pageSize.getHeight();
        const maxWidth = pageWidth - marginX * 2;
        let y = 60;

        doc.setFont('helvetica', 'bold');
        doc.setFontSize(20);
        const titleLines = doc.splitTextToSize(node.title || 'Untitled Note', maxWidth);
        titleLines.forEach((line: string) => {
          if (y > pageHeight - 50) { doc.addPage(); y = 60; }
          doc.text(line, marginX, y);
          y += 24;
        });
        y += 16;

        doc.setFont('helvetica', 'normal');
        doc.setFontSize(12);
        const bodyText = stripHtml(node.content) || ' ';
        const bodyLines = doc.splitTextToSize(bodyText, maxWidth);
        const lineHeight = 18;

        bodyLines.forEach((line: string) => {
          if (y > pageHeight - 50) {
            doc.addPage();
            y = 60;
          }
          doc.text(line, marginX, y);
          y += lineHeight;
        });

        const fileName = (node.title || 'note').replace(/[^a-z0-9\-_ ]/gi, '').trim() || 'note';
        doc.save(`${fileName}.pdf`);
        showToast('PDF downloaded');
      } catch (e) {
        showToast('Could not generate PDF');
      }
      setLongPressId(null);
    };

    // Move note to Recycle Bin (Trash)
    const handleDeleteNote = (id: string) => {
      if (!tree) return;
      const nextTree = cloneTree(tree);
      const parent = findParentNode(nextTree, id);
      const nodeToDelete = findNode(nextTree, id);
      if (!nodeToDelete) return;

      deleteNodeById(nextTree, id);
```

src/App.tsx (continued from previous page)

```
273 |       setTree(nextTree);
274 |
275 |       setTrash((prev) => [
276 |         {
277 |           id: generateUid(),
278 |           node: cloneTree(nodeToDelete),
279 |           parentId: parent ? parent.id : 'root',
280 |           deletedAt: Date.now(),
281 |         },
282 |         ...prev,
283 |       ]);
284 |
285 |       setLongPressId(null);
286 |       showToast('Moved note to Recycle Bin');
287 |     };
288 |
289 |     // Restore Note from Recycle Bin to original parent page
290 |     const handleRestoreFromTrash = (trashId: string) => {
291 |       const entry = trash.find((t) => t.id === trashId);
292 |       if (!entry || !tree) return;
293 |
294 |       const nextTree = cloneTree(tree);
295 |       let parent = findNode(nextTree, entry.parentId);
296 |       if (!parent) parent = nextTree;
297 |
298 |       const restoredNode = cloneTree(entry.node);
299 |       if (!parent.children) parent.children = [];
300 |       parent.children.push(restoredNode);
301 |
302 |       // Re-embed chip placeholder into parent note writing content if missing
303 |       if (!contentHasChip(parent.content, restoredNode.id)) {
304 |         parent.content = (parent.content || '') + makeChipHtml(restoredNode.id, restoredNode.title,
    > |restoredNode.pinned) + '<br>';
305 |       }
306 |
307 |       setTree(nextTree);
308 |       setTrash((prev) => prev.filter((t) => t.id !== trashId));
309 |
310 |       // When parent page is clicked/navigated to, lands on parent writing page containing the
    > |restored note chip
311 |       showToast(restoredNode.pinned ? 'Restored pinned note to parent page' : 'Restored note to
    > |parent page');
312 |     };
313 |
314 |     const handlePermanentDelete = (trashId: string) => {
315 |       setTrash((prev) => prev.filter((t) => t.id !== trashId));
316 |       showToast('Permanently deleted');
317 |     };
318 |
319 |     // Archive Note
320 |     const handleArchiveNote = (id: string) => {
321 |       updateTree((t) => {
322 |         const node = findNode(t, id);
323 |         if (node) {
324 |           node.archived = true;
325 |         }
326 |       });
327 |       setLongPressId(null);
328 |       showToast('Archived note');
329 |     };
330 |
331 |     // Restore Archived Note
332 |     const handleRestoreArchived = (id: string) => {
333 |       updateTree((t) => {
334 |         const node = findNode(t, id);
335 |         if (node) {
336 |           node.archived = false;
337 |           const parent = findParentNode(t, id);
338 |           if (parent && !contentHasChip(parent.content, id)) {
```

src/App.tsx (continued from previous page)

```
339 |             parent.content = (parent.content || '') + makeChipHtml(id, node.title, node.pinned) +
    > |'<br>';
340 |           }
341 |         }
342 |       });
343 |       showToast('Restored note to parent page');
344 |     };
345 |
346 |     // Title / Content updates
347 |     const handleTitleChange = (newTitle: string) => {
348 |       updateTree((t) => {
349 |         const node = findNode(t, currentId);
350 |         if (node) {
351 |           node.title = newTitle;
352 |           node.updatedAt = Date.now();
353 |         }
354 |       });
355 |     };
356 |
357 |     const handleContentChange = (newHtml: string) => {
358 |       updateTree((t) => {
359 |         const node = findNode(t, currentId);
360 |         if (node) {
361 |           node.content = newHtml;
362 |           node.updatedAt = Date.now();
363 |         }
364 |       });
365 |     };
366 |
367 |     const handleUpdateNodeContentDirect = (id: string, title: string, content: string) => {
368 |       updateTree((t) => {
369 |         const node = findNode(t, id);
370 |         if (node) {
371 |           node.title = title;
372 |           node.content = content;
373 |           node.updatedAt = Date.now();
374 |         }
375 |       });
376 |     };
377 |
378 |     // Search result selection - builds path stack to parent writing page
379 |     const handleSelectSearchResult = (result: any) => {
380 |       setStack(result.path);
381 |       setMode('write');
382 |       setSearchOpen(false);
383 |     };
384 |
385 |     if (loadError) {
386 |       return <div className="pb-shell pb-loading"><span>Saved notes could not be read. Your stored
    > |data has been left unchanged. Back it up before repairing the storage record, then
    > |reload.</span></div>;
387 |     }
388 |
389 |     if (!loaded || !tree || !currentNode) {
390 |       return (
391 |         <div className="pb-shell pb-loading">
392 |           <span>Loading Pinboard Notes...</span>
393 |         </div>
394 |       );
395 |     }
396 |
397 |     return (
398 |       <div className="pb-shell">
399 |         {/* Top Header */}
400 |         <TopBar
401 |           mode={mode}
402 |           currentNode={currentNode}
403 |           stack={stack}
404 |           theme={theme}
```

src/App.tsx (continued from previous page)

```
405 |           trashAreaNode={trashAreaNode}
406 |           onSetMode={setMode}
407 |           onBack={handleBack}
408 |           onToggleTheme={() => setTheme(theme === 'dark' ? 'light' : 'dark')}
409 |           onOpenSearch={() => setSearchOpen(true)}
410 |           onTitleChange={handleTitleChange}
411 |           onOpenTrashForArea={() => {
412 |             setTrashAreaId(currentId);
413 |             setMode('trash');
414 |           }}
415 |           onOpenCodeExport={() => setCodeExportOpen(true)}
416 |         />
417 |
418 |         {/* Main Board View */}
419 |         {mode === 'board' && (
420 |           <BoardView
421 |             currentNode={currentNode}
422 |             onOpenNote={handleOpenNote}
423 |             onLongPressNote={setLongPressId}
424 |             onAddChild={handleAddChild}
425 |             onUpdateTree={updateTree}
426 |           />
427 |         )}
428 |
429 |         {/* Main Editor View */}
430 |         {mode === 'write' && (
431 |           <EditorView
432 |             currentNode={currentNode}
433 |             tree={tree}
434 |             stack={stack}
435 |             onBack={handleBack}
436 |             onSetModeBoard={() => setMode('board')}
437 |             onTitleChange={handleTitleChange}
438 |             onContentChange={handleContentChange}
439 |             onInsertNestedNote={handleInsertNestedNote}
440 |             onPopOutBubble={() => setFloatingId(currentId)}
441 |             onNavigateToNote={handleOpenNote}
442 |             onLongPressNote={setLongPressId}
443 |             onShowToast={showToast}
444 |             onOpenCodeExport={() => setCodeExportOpen(true)}
445 |             onDownloadCurrentNotePdf={() => currentNode && handleDownloadPdf(currentNode)}
446 |           />
447 |         )}
448 |
449 |         {/* Archive & Recycle Bin Views */}
450 |         {(mode === 'archive' || mode === 'trash') && (
451 |           <ArchiveTrashView
452 |             mode={mode}
453 |             currentNode={currentNode}
454 |             tree={tree}
455 |             trash={trash}
456 |             trashAreaId={trashAreaId}
457 |             onRestoreArchived={handleRestoreArchived}
458 |             onRestoreTrash={handleRestoreFromTrash}
459 |             onPermanentDelete={handlePermanentDelete}
460 |             onDeleteNote={handleDeleteNote}
461 |           />
462 |         )}
463 |
464 |         {/* Editable Floating Bubble Pin */}
465 |         <BubblePin
466 |           floatingNode={floatingNode}
467 |           settings={bubbleSettings}
468 |           onUpdateSettings={(newSettings) =>
469 |             setBubbleSettings((prev) => ({ ...prev, ...newSettings }))
470 |           }
471 |           onUpdateNodeContent={handleUpdateNodeContentDirect}
472 |           onCopyNote={handleCopyNote}
473 |           onShowToast={showToast}
```

src/App.tsx (continued from previous page)

```
474 |           onClose={() => setFloatingId(null)}
475 |         />
476 |
477 |         {/* Long-Press Action Sheet */}
478 |         <ActionSheet
479 |           longPressNode={longPressNode}
480 |           onClose={() => setLongPressId(null)}
481 |           onTogglePin={handleTogglePin}
482 |           onCopyNote={handleCopyNote}
483 |           onDownloadPdf={handleDownloadPdf}
484 |           onArchive={handleArchiveNote}
485 |           onDelete={handleDeleteNote}
486 |           onOpenCodeExport={() => setCodeExportOpen(true)}
487 |         />
488 |
489 |         {/* Global Search Modal */}
490 |         <SearchModal
491 |           tree={tree}
492 |           isOpen={searchOpen}
493 |           onClose={() => setSearchOpen(false)}
494 |           onSelectResult={handleSelectSearchResult}
495 |         />
496 |
497 |         {/* Codebase PDF & Source Export Modal */}
498 |         <CodeExportModal
499 |           isOpen={codeExportOpen}
500 |           onClose={() => setCodeExportOpen(false)}
501 |           onShowToast={showToast}
502 |         />
503 |
504 |         {/* Notification Toast */}
505 |         {(saveError || toast) && <div className="toast">{saveError ? 'Could not save notes. Keep
    > |this page open and free browser storage.' : toast}</div>}
506 |       </div>
507 |     );
508 |   }
509 |
```

src/index.css

Design System | 1607 Lines

Role: Complete CSS variables, dark/light themes, and animation rules

```
@import "tailwindcss";

/* The original font families, served locally for reliable static hosting. */
@font-face {
  font-family: 'Inter';
  font-style: normal;
  font-weight: 100 900;
  font-display: swap;
  src: url('../public/fonts/Inter-variable.ttf') format('truetype');
}
@font-face {
  font-family: 'Plus Jakarta Sans';
  font-style: normal;
  font-weight: 200 800;
  font-display: swap;
  src: url('../public/fonts/PlusJakartaSans-variable.ttf') format('truetype');
}

* {
  box-sizing: border-box;
}

/* Theme variables */
:root {
  --bg1: #0A1024;
  --bg2: #131B3B;
  --bg3: #1B2657;
  --dot: rgba(255, 255, 255, 0.05);
  --text: #EDEFF7;
  --text-dim: #A9B0CE;
  --text-faint: #6B759C;
  --placeholder: #545E86;
  --card-bg: rgba(255, 255, 255, 0.045);
  --card-bg-hover: rgba(255, 255, 255, 0.075);
  --card-border: rgba(255, 255, 255, 0.09);
  --icon-bg: rgba(255, 255, 255, 0.05);
  --icon-border: rgba(255, 255, 255, 0.09);
  --icon-bg-hover: rgba(255, 255, 255, 0.12);
  --accent1: #3A5FE0;
  --accent2: #1E3FA0;
  --accent-shadow: rgba(46, 79, 224, 0.45);
  --pill-bg: rgba(58, 95, 224, 0.16);
  --pill-border: rgba(94, 127, 255, 0.35);
  --pill-text: #9CB0FF;
  --divider: rgba(255, 255, 255, 0.09);
  --sheet-bg: #171F45;
  --sheet-border: rgba(255, 255, 255, 0.12);
  --danger: #FF8B8B;
  --danger-bg: rgba(224, 58, 58, 0.16);
  --pin-glow: rgba(255, 196, 84, 0.35);
  --pin-border: rgba(255, 201, 102, 0.55);
  --pin-active: #FFC966;
  --toast-bg: rgba(23, 31, 69, 0.95);
  --toolbar-bg: rgba(19, 27, 59, 0.92);
  --caret: #7C9BFF;
  --glow-color: rgba(108, 140, 255, 0.18);
  --badge-ring: #131B3B;
  --highlight-bg: #FFC966;
  --highlight-text: #1A1024;
}

[data-theme="light"] {
  --bg1: #F3EAD9;
  --bg2: #F6EFE1;
  --bg3: #EFE1C8;
```

src/index.css (continued from previous page)

```
    --dot: rgba(32, 36, 28, 0.045);
    --text: #20241C;
    --text-dim: #5B5748;
    --text-faint: #9A9280;
    --placeholder: #BDB39C;
    --card-bg: rgba(255, 255, 255, 0.76);
    --card-bg-hover: rgba(255, 255, 255, 0.96);
    --card-border: rgba(32, 36, 28, 0.09);
    --icon-bg: rgba(32, 36, 28, 0.05);
    --icon-border: rgba(32, 36, 28, 0.09);
    --icon-bg-hover: rgba(32, 36, 28, 0.12);
    --accent1: #E8823D;
    --accent2: #C9631F;
    --accent-shadow: rgba(232, 130, 61, 0.40);
    --pill-bg: rgba(76, 91, 62, 0.12);
    --pill-border: rgba(76, 91, 62, 0.35);
    --pill-text: #4C5B3E;
    --divider: rgba(32, 36, 28, 0.08);
    --sheet-bg: #FBF5EA;
    --sheet-border: rgba(32, 36, 28, 0.12);
    --danger: #C0392B;
    --danger-bg: rgba(192, 57, 43, 0.10);
    --pin-glow: rgba(232, 130, 61, 0.35);
    --pin-border: rgba(232, 130, 61, 0.55);
    --pin-active: #C9631F;
    --toast-bg: rgba(32, 36, 28, 0.92);
    --toolbar-bg: rgba(246, 239, 225, 0.94);
    --caret: #C9631F;
    --glow-color: rgba(232, 130, 61, 0.18);
    --badge-ring: #F6EFE1;
    --highlight-bg: #E8823D;
    --highlight-text: #FBF5EA;
  }

  body {
    margin: 0;
    padding: 0;
    overflow: hidden;
    font-family: 'Inter', sans-serif;
    background: var(--bg1);
    color: var(--text);
  }

  .pb-shell {
    height: 100vh;
    height: 100dvh;
    width: 100%;
    display: flex;
    flex-direction: column;
    position: relative;
    overflow: hidden;
    background:
      radial-gradient(circle at 1px 1px, var(--dot) 1px, transparent 1px),
      linear-gradient(160deg, var(--bg1) 0%, var(--bg2) 45%, var(--bg3) 100%);
    background-size: 7px 7px, 100% 100%;
    transition: background 0.4s ease, color 0.4s ease;
  }

  .pb-loading {
    display: flex;
    align-items: center;
    justify-content: center;
    color: var(--text-faint);
    font-weight: 600;
  }

  /* Header & TopBar */
  .pb-topbar {
    display: flex;
```

src/index.css (continued from previous page)

```
    flex-direction: column;
    gap: 10px;
    padding: 16px 18px 12px 18px;
    flex-shrink: 0;
    position: relative;
    z-index: 5;
  }

  .pb-topbar-icons {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 8px;
  }

  .pb-topbar-icons-left,
  .pb-topbar-icons-right {
    display: flex;
    align-items: center;
    gap: 8px;
  }

  .icon-btn {
    width: 40px;
    height: 40px;
    border-radius: 13px;
    border: 1px solid var(--icon-border);
    background: var(--icon-bg);
    backdrop-filter: blur(8px);
    display: flex;
    align-items: center;
    justify-content: center;
    cursor: pointer;
    color: var(--text-dim);
    flex-shrink: 0;
    transition: all 0.15s ease;
  }

  .icon-btn:hover {
    background: var(--icon-bg-hover);
    color: var(--text);
  }

  .icon-btn.mini {
    width: 32px;
    height: 32px;
    border-radius: 9px;
  }

  .icon-btn.active-accent {
    background: var(--pill-bg);
    border-color: var(--pill-border);
    color: var(--pill-text);
  }

  .pb-title-row {
    display: flex;
  }

  .pb-title-input {
    flex: 1;
    width: 100%;
    font-family: 'Plus Jakarta Sans', sans-serif;
    font-weight: 800;
    font-size: 22px;
    border: none;
    background: transparent;
    outline: none;
    color: var(--text);
```

src/index.css (continued from previous page)

```
  }

  /* Board View & Sections */
  .board-scroll {
    flex: 1;
    overflow-y: auto;
    position: relative;
    z-index: 2;
  }

  .board-inner {
    max-width: 640px;
    margin: 0 auto;
    padding: 8px 18px 90px 18px;
    display: flex;
    flex-direction: column;
    gap: 20px;
  }

  .board-empty {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    gap: 10px;
    padding-top: 100px;
    color: var(--text-faint);
    text-align: center;
  }

  .empty-icon-circle {
    width: 64px;
    height: 64px;
    border-radius: 50%;
    background: var(--card-bg);
    border: 1px solid var(--card-border);
    display: flex;
    align-items: center;
    justify-content: center;
    color: var(--text-dim);
  }

  .board-empty p {
    font-family: 'Plus Jakarta Sans', sans-serif;
    font-weight: 700;
    font-size: 16px;
    margin: 0;
    color: var(--text);
  }

  .empty-sub {
    font-size: 13px;
    color: var(--text-faint);
  }

  /* Section headers for hierarchy of importance */
  .section-block {
    display: flex;
    flex-direction: column;
    gap: 12px;
  }

  .section-header {
    display: flex;
    align-items: center;
    gap: 8px;
    font-family: 'Plus Jakarta Sans', sans-serif;
    font-weight: 800;
    font-size: 11.5px;
```

src/index.css (continued from previous page)

```
    letter-spacing: 0.08em;
    color: var(--pin-active);
  }

  .pin-hdr-icon {
    color: var(--pin-active);
  }

  .section-hdr-line {
    flex: 1;
    height: 1px;
    background: var(--divider);
  }

  .cards-grid {
    display: flex;
    flex-direction: column;
    gap: 14px;
  }

  /* Note Cards with Hold to Reorder */
  .note-card {
    position: relative;
    border-radius: 20px;
    padding: 20px 20px 18px 20px;
    background: var(--card-bg);
    border: 1px solid var(--card-border);
    backdrop-filter: blur(10px);
    box-shadow: 0 8px 24px rgba(0, 0, 0, 0.12);
    cursor: pointer;
    user-select: none;
    transition: transform 0.18s ease, background 0.18s ease, box-shadow 0.25s ease;
  }

  .note-card:hover {
    background: var(--card-bg-hover);
  }

  .note-card:active {
    transform: scale(0.985);
  }

  .note-card.dragging {
    opacity: 0.5;
    transform: scale(0.96);
  }

  .note-card.pinned-glow {
    border-color: var(--pin-border);
    box-shadow: 0 0 0 1px var(--pin-border), 0 0 24px var(--pin-glow), 0 8px 24px rgba(0,0,0,0.12);
  }

  .card-drag-handle {
    position: absolute;
    top: 16px;
    right: 16px;
    color: var(--text-faint);
    opacity: 0.4;
    cursor: grab;
    padding: 4px;
  }

  .card-drag-handle:hover {
    opacity: 0.9;
    color: var(--text);
  }

  .pin-emoji {
    position: absolute;
```

src/index.css (continued from previous page)

```
    top: 16px;
    left: 16px;
    font-size: 13px;
    opacity: 0.3;
    transform: rotate(-15deg);
    transition: all 0.2s ease;
  }

  .pin-emoji-active {
    opacity: 1;
    filter: drop-shadow(0 0 6px var(--pin-glow));
  }

  .card-pill {
    display: inline-block;
    font-size: 10px;
    font-weight: 800;
    letter-spacing: 0.08em;
    color: var(--pill-text);
    background: var(--pill-bg);
    border: 1px solid var(--pill-border);
    border-radius: 20px;
    padding: 3px 9px;
    margin: 0 0 10px 22px;
  }

  .card-title {
    font-family: 'Plus Jakarta Sans', sans-serif;
    font-weight: 800;
    font-size: 20px;
    color: var(--text);
    margin-bottom: 10px;
  }

  .card-divider {
    height: 1px;
    background: var(--divider);
    margin-bottom: 10px;
  }

  .card-body {
    font-size: 13.5px;
    line-height: 1.6;
    color: var(--text-dim);
  }

  .count-badge {
    position: absolute;
    top: -8px;
    right: 42px;
    min-width: 26px;
    height: 26px;
    border-radius: 50%;
    background: linear-gradient(160deg, var(--accent1), var(--accent2));
    color: #fff;
    font-family: 'Plus Jakarta Sans', sans-serif;
    font-weight: 700;
    font-size: 12px;
    display: flex;
    align-items: center;
    justify-content: center;
    border: 2px solid var(--badge-ring);
    box-shadow: 0 4px 10px rgba(0,0,0,0.25);
  }

  /* Floating Action Button (Create Pin +) */
  .fab-add {
    position: fixed;
    right: calc(20px + env(safe-area-inset-right, 0px));
```

src/index.css (continued from previous page)

```
    bottom: calc(24px + env(safe-area-inset-bottom, 0px));
    width: 58px;
    height: 58px;
    border-radius: 50%;
    border: none;
    background: linear-gradient(160deg, var(--accent1), var(--accent2));
    color: #fff;
    display: flex;
    align-items: center;
    justify-content: center;
    cursor: pointer;
    box-shadow: 0 10px 26px var(--accent-shadow);
    z-index: 8;
    transition: transform 0.18s ease, filter 0.18s ease;
  }

  .fab-add:hover {
    filter: brightness(1.1);
    transform: scale(1.04);
  }

  .fab-add:active {
    transform: scale(0.94);
  }

  /* Writer / Editor View */
  .write-scroll {
    flex: 1;
    overflow-y: auto;
    position: relative;
    z-index: 2;
    cursor: text;
  }

  .write-page {
    max-width: 650px;
    margin: 0 auto;
    padding: 66px 22px 160px 22px;
    border-radius: 18px;
    min-height: 100%;
    display: flex;
    flex-direction: column;
    transition: box-shadow 0.3s ease;
  }

  .write-page.glow-focus {
    box-shadow: 0 0 0 1px var(--glow-color), 0 0 40px var(--glow-color);
  }

  .write-float-back {
    position: fixed;
    top: calc(14px + env(safe-area-inset-top, 0px));
    left: 14px;
    z-index: 10;
    backdrop-filter: blur(10px);
    box-shadow: 0 6px 16px rgba(0,0,0,0.18);
  }

  .write-float-grid {
    position: fixed;
    top: calc(14px + env(safe-area-inset-top, 0px));
    right: 14px;
    z-index: 10;
    backdrop-filter: blur(10px);
    box-shadow: 0 6px 16px rgba(0,0,0,0.18);
  }

  .write-title-input {
    width: 100%;
```

src/index.css (continued from previous page)

```
480 |     border: none;
481 |     background: transparent;
482 |     outline: none;
483 |     font-family: 'Plus Jakarta Sans', sans-serif;
484 |     font-weight: 800;
485 |     font-size: 24px;
486 |     color: var(--text);
487 |     margin-bottom: 14px;
488 |     flex-shrink: 0;
489 |   }
490 |
491 |   .write-title-input::placeholder {
492 |     color: var(--placeholder);
493 |   }
494 |
495 |   .write-content {
496 |     font-family: 'Inter', sans-serif;
497 |     font-size: 16px;
498 |     line-height: 1.7;
499 |     flex: 1;
500 |     min-height: 320px;
501 |     outline: none;
502 |     color: var(--text);
503 |     caret-color: var(--caret);
504 |   }
505 |
506 |   .write-content:empty:before {
507 |     content: 'Start typing note...';
508 |     color: var(--placeholder);
509 |   }
510 |
511 |   .write-content h1 {
512 |     font-family: 'Plus Jakarta Sans', sans-serif;
513 |     font-size: 26px;
514 |     font-weight: 800;
515 |     margin: 0.5em 0 0.3em;
516 |     color: var(--text);
517 |   }
518 |
519 |   .write-content h2 {
520 |     font-family: 'Plus Jakarta Sans', sans-serif;
521 |     font-size: 20px;
522 |     font-weight: 700;
523 |     margin: 0.5em 0 0.3em;
524 |     color: var(--text);
525 |   }
526 |
527 |   .write-content ul, .write-content ol {
528 |     padding-left: 1.3em;
529 |   }
530 |
531 |   .write-content img {
532 |     max-width: 100%;
533 |     border-radius: 12px;
534 |     margin: 10px 0;
535 |   }
536 |
537 |   /* Embedded Pinned Note Chips inside Editor */
538 |   .write-content .note-chip {
539 |     display: inline-flex;
540 |     align-items: center;
541 |     gap: 8px;
542 |     width: fit-content;
543 |     max-width: 100%;
544 |     margin: 10px 0;
545 |     padding: 8px 14px;
546 |     border-radius: 14px;
547 |     background: var(--pill-bg);
548 |     border: 1px solid var(--pill-border);
```

src/index.css (continued from previous page)

```
549 |     cursor: pointer;
550 |     user-select: none;
551 |     transition: all 0.2s ease;
552 |   }
553 |
554 |   .write-content .note-chip:hover {
555 |     background: var(--icon-bg-hover);
556 |   }
557 |
558 |   .write-content .note-chip.note-chip-pinned {
559 |     border-color: var(--pin-border);
560 |     box-shadow: 0 0 0 1px var(--pin-border), 0 0 18px var(--pin-glow);
561 |   }
562 |
563 |   .write-content .note-chip.note-chip-missing {
564 |     opacity: 0.5;
565 |   }
566 |
567 |   .write-content .note-chip-pin::before {
568 |     content: "•";
569 |     font-size: 13px;
570 |     color: var(--text-faint);
571 |   }
572 |
573 |   .write-content .note-chip.note-chip-pinned .note-chip-pin::before {
574 |       c o n t e n t :   "Ø=ÜÌ " ;
575 |     filter: drop-shadow(0 0 6px var(--pin-glow));
576 |   }
577 |
578 |   .write-content .note-chip-label {
579 |     font-family: 'Plus Jakarta Sans', sans-serif;
580 |     font-weight: 700;
581 |     font-size: 14px;
582 |     color: var(--pill-text);
583 |   }
584 |
585 |   /* Toolbar & Unique Bubble Pop Button */
586 |   .write-toolbar {
587 |     position: fixed;
588 |     bottom: 0;
589 |     left: 0;
590 |     right: 0;
591 |     display: flex;
592 |     align-items: center;
593 |     justify-content: safe center;
594 |     gap: 4px;
595 |     padding: 10px 14px calc(10px + env(safe-area-inset-bottom, 0px)) 14px;
596 |     background: var(--toolbar-bg);
597 |     backdrop-filter: blur(14px);
598 |     border-top: 1px solid var(--divider);
599 |     z-index: 6;
600 |     overflow-x: auto;
601 |   }
602 |
603 |   .tb-btn {
604 |     width: 38px;
605 |     height: 38px;
606 |     border-radius: 11px;
607 |     border: none;
608 |     background: var(--icon-bg);
609 |     display: flex;
610 |     align-items: center;
611 |     justify-content: center;
612 |     cursor: pointer;
613 |     color: var(--text-dim);
614 |     flex-shrink: 0;
615 |     transition: all 0.15s ease;
616 |   }
617 |
```

src/index.css (continued from previous page)

```
  .tb-btn:hover {
    background: var(--icon-bg-hover);
    color: var(--text);
  }

  .tb-btn.accent-pin {
    background: linear-gradient(160deg, var(--accent1), var(--accent2));
    color: #fff;
    box-shadow: 0 4px 12px var(--accent-shadow);
  }

  /* Standout Unique Bubble Pop Button */
  .tb-btn.unique-bubble-pop {
    width: auto;
    padding: 0 14px;
    height: 38px;
    border-radius: 20px;
    border: 1px solid var(--pin-border);
    background: linear-gradient(135deg, var(--pin-active) 0%, var(--accent1) 100%);
    color: #1A1024;
    font-family: 'Plus Jakarta Sans', sans-serif;
    font-weight: 800;
    font-size: 13px;
    display: flex;
    align-items: center;
    gap: 6px;
    box-shadow: 0 0 16px var(--pin-glow), 0 4px 12px rgba(0,0,0,0.25);
    cursor: pointer;
    animation: floatPulse 3s ease-in-out infinite;
  }

  .tb-btn.unique-bubble-pop:hover {
    filter: brightness(1.1);
    transform: scale(1.03);
  }

  @keyframes floatPulse {
    0%, 100% { box-shadow: 0 0 14px var(--pin-glow), 0 4px 12px rgba(0,0,0,0.25); }
    50% { box-shadow: 0 0 24px var(--pin-glow), 0 6px 18px rgba(0,0,0,0.3); }
  }

  .sparkle-spin {
    animation: spinSlow 6s linear infinite;
  }

  @keyframes spinSlow {
    from { transform: rotate(0deg); }
    to { transform: rotate(360deg); }
  }

  .tb-sep {
    width: 1px;
    height: 24px;
    background: var(--divider);
    margin: 0 4px;
    flex-shrink: 0;
  }

  /* Floating Bubble Pin (Unique Design) */
  .float-bubble-unique {
    position: fixed;
    border-radius: 50%;
    background: linear-gradient(135deg, var(--pin-active) 0%, var(--accent1) 100%);
    box-shadow: 0 0 24px var(--pin-glow), 0 10px 26px var(--accent-shadow);
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    color: #1A1024;
```

src/index.css (continued from previous page)

```
    z-index: 50;
    cursor: grab;
    touch-action: none;
    user-select: none;
    padding: 4px;
    text-align: center;
    transition: opacity 0.2s ease, transform 0.15s ease;
  }

  .float-bubble-unique:active {
    cursor: grabbing;
    transform: scale(0.95);
  }

  .bubble-glow-ring {
    position: absolute;
    inset: -3px;
    border-radius: 50%;
    border: 2px dashed var(--pin-border);
    opacity: 0.6;
    animation: spinSlow 12s linear infinite;
  }

  .float-bubble-pin-icon {
    font-size: 16px;
  }

  .float-bubble-title-text {
    font-family: 'Plus Jakarta Sans', sans-serif;
    font-size: 9px;
    font-weight: 800;
    margin-top: 2px;
    max-width: 50px;
    overflow: hidden;
    text-overflow: ellipsis;
    white-space: nowrap;
  }

  /* Floating Editable Panel & Settings Customizer */
  .float-panel-editable {
    position: fixed;
    right: 18px;
    bottom: 24px;
    width: 90vw;
    background: var(--sheet-bg);
    border: 1px solid var(--sheet-border);
    border-radius: 22px;
    box-shadow: 0 20px 60px rgba(0, 0, 0, 0.4);
    z-index: 50;
    display: flex;
    flex-direction: column;
    overflow: hidden;
    backdrop-filter: blur(16px);
    transition: opacity 0.2s ease;
  }

  .float-panel-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 10px;
    padding: 12px 16px;
    border-bottom: 1px solid var(--divider);
    flex-shrink: 0;
  }

  .float-panel-title-input {
    flex: 1;
    border: none;
```

src/index.css (continued from previous page)

```
    background: transparent;
    outline: none;
    font-family: 'Plus Jakarta Sans', sans-serif;
    font-weight: 800;
    font-size: 15px;
    color: var(--text);
    min-width: 0;
  }

  .float-panel-actions {
    display: flex;
    align-items: center;
    gap: 6px;
  }

  /* Settings Tray inside Floating Panel */
  .bubble-settings-tray {
    background: var(--card-bg);
    border-bottom: 1px solid var(--divider);
    padding: 12px 16px;
    display: flex;
    flex-direction: column;
    gap: 12px;
    font-size: 12.5px;
  }

  .tray-title {
    display: flex;
    align-items: center;
    justify-content: space-between;
    font-family: 'Plus Jakarta Sans', sans-serif;
    font-weight: 800;
    color: var(--pin-active);
  }

  .reset-btn {
    border: none;
    background: transparent;
    color: var(--text-dim);
    font-size: 11px;
    cursor: pointer;
    display: flex;
    align-items: center;
    gap: 4px;
  }

  .setting-group {
    display: flex;
    flex-direction: column;
    gap: 6px;
  }

  .setting-group label {
    font-weight: 600;
    color: var(--text-dim);
  }

  .setting-dual {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 10px;
  }

  .setting-dual span {
    font-size: 11px;
    color: var(--text-faint);
  }

  input[type="range"] {
```

src/index.css (continued from previous page)

```
    width: 100%;
    accent-color: var(--accent1);
  }

  .preset-pills {
    display: flex;
    gap: 8px;
  }

  .preset-pill {
    flex: 1;
    padding: 4px;
    border-radius: 8px;
    border: 1px solid var(--card-border);
    background: var(--icon-bg);
    color: var(--text);
    font-weight: 700;
    font-size: 12px;
    cursor: pointer;
  }

  .preset-pill.selected {
    background: var(--pill-bg);
    border-color: var(--pill-border);
    color: var(--pill-text);
  }

  .float-panel-scroll-container {
    flex: 1;
    overflow-y: auto;
    padding: 14px 16px;
  }

  .float-panel-scroll-container.cropped-mask {
    overflow: hidden;
    border: 1px dashed var(--pin-border);
    border-radius: 12px;
    margin: 8px;
  }

  .float-panel-body-editable {
    outline: none;
    font-size: 14.5px;
    line-height: 1.6;
    color: var(--text);
    min-height: 180px;
  }

  /* Action Sheet */
  .sheet-backdrop {
    position: fixed;
    inset: 0;
    background: rgba(4, 7, 20, 0.45);
    backdrop-filter: blur(3px);
    display: flex;
    align-items: flex-end;
    justify-content: center;
    z-index: 30;
  }

  .sheet {
    width: 100%;
    max-width: 500px;
    background: var(--sheet-bg);
    border: 1px solid var(--sheet-border);
    border-radius: 22px 22px 0 0;
    padding: 10px 12px calc(16px + env(safe-area-inset-bottom, 0px)) 12px;
    box-shadow: 0 -10px 40px rgba(0,0,0,0.35);
  }
```

src/index.css (continued from previous page)

```
894 |
895 |   .sheet-title {
896 |     font-family: 'Plus Jakarta Sans', sans-serif;
897 |     font-weight: 800;
898 |     font-size: 15px;
899 |     color: var(--text);
900 |     text-align: center;
901 |     padding: 12px 8px;
902 |     border-bottom: 1px solid var(--divider);
903 |     margin-bottom: 6px;
904 |   }
905 |
906 |   .sheet-item {
907 |     width: 100%;
908 |     padding: 13px 12px;
909 |     border-radius: 12px;
910 |     border: none;
911 |     background: transparent;
912 |     color: var(--text);
913 |     font-weight: 600;
914 |     font-size: 14.5px;
915 |     cursor: pointer;
916 |     display: flex;
917 |     align-items: center;
918 |     gap: 12px;
919 |     transition: background 0.15s ease;
920 |   }
921 |
922 |   .sheet-item:hover {
923 |     background: var(--icon-bg);
924 |   }
925 |
926 |   .sheet-item.active-pin {
927 |     color: var(--pin-active);
928 |   }
929 |
930 |   .sheet-item.danger {
931 |     color: var(--danger);
932 |   }
933 |
934 |   .sheet-item.danger:hover {
935 |     background: var(--danger-bg);
936 |   }
937 |
938 |   .sheet-cancel {
939 |     width: 100%;
940 |     padding: 13px;
941 |     border-radius: 12px;
942 |     border: none;
943 |     background: var(--icon-bg);
944 |     color: var(--text-dim);
945 |     font-weight: 600;
946 |     font-size: 14.5px;
947 |     cursor: pointer;
948 |     margin-top: 6px;
949 |   }
950 |
951 |   /* Search Modal */
952 |   .search-backdrop {
953 |     position: fixed;
954 |     inset: 0;
955 |     background: rgba(4, 7, 20, 0.45);
956 |     backdrop-filter: blur(3px);
957 |     display: flex;
958 |     align-items: flex-start;
959 |     justify-content: center;
960 |     z-index: 35;
961 |     padding-top: max(16px, env(safe-area-inset-top, 0px));
962 |   }
```

src/index.css (continued from previous page)

```
963 |
964 |   .search-panel {
965 |     width: 100%;
966 |     max-width: 560px;
967 |     max-height: min(78vh, 640px);
968 |     margin: 0 14px;
969 |     background: var(--sheet-bg);
970 |     border: 1px solid var(--sheet-border);
971 |     border-radius: 20px;
972 |     box-shadow: 0 20px 60px rgba(0,0,0,0.35);
973 |     display: flex;
974 |     flex-direction: column;
975 |     overflow: hidden;
976 |   }
977 |
978 |   .search-input-row {
979 |     display: flex;
980 |     align-items: center;
981 |     gap: 10px;
982 |     padding: 14px 16px;
983 |     border-bottom: 1px solid var(--divider);
984 |   }
985 |
986 |   .search-input {
987 |     flex: 1;
988 |     border: none;
989 |     background: transparent;
990 |     outline: none;
991 |     font-size: 15.5px;
992 |     color: var(--text);
993 |   }
994 |
995 |   .search-close-btn {
996 |     width: 32px;
997 |     height: 32px;
998 |     border-radius: 9px;
999 |     border: none;
1000 |     background: var(--icon-bg);
1001 |     color: var(--text-dim);
1002 |     display: flex;
1003 |     align-items: center;
1004 |     justify-content: center;
1005 |     cursor: pointer;
1006 |   }
1007 |
1008 |   .search-results {
1009 |     overflow-y: auto;
1010 |     padding: 8px 10px 14px 10px;
1011 |   }
1012 |
1013 |   .search-empty {
1014 |     padding: 40px 20px;
1015 |     text-align: center;
1016 |     color: var(--text-faint);
1017 |     font-size: 14px;
1018 |   }
1019 |
1020 |   .search-result-item {
1021 |     width: 100%;
1022 |     text-align: left;
1023 |     padding: 12px;
1024 |     border-radius: 12px;
1025 |     border: none;
1026 |     background: transparent;
1027 |     cursor: pointer;
1028 |     margin-bottom: 2px;
1029 |   }
1030 |
1031 |   .search-result-item:hover {
```

src/index.css (continued from previous page)

```
1032 |     background: var(--icon-bg);
1033 |   }
1034 |
1035 |   .search-result-head {
1036 |     display: flex;
1037 |     align-items: center;
1038 |     gap: 8px;
1039 |     margin-bottom: 4px;
1040 |   }
1041 |
1042 |   .search-result-title {
1043 |     font-family: 'Plus Jakarta Sans', sans-serif;
1044 |     font-weight: 700;
1045 |     font-size: 14.5px;
1046 |     color: var(--text);
1047 |     flex: 1;
1048 |   }
1049 |
1050 |   .search-result-tag {
1051 |     font-size: 9.5px;
1052 |     font-weight: 800;
1053 |     color: var(--pill-text);
1054 |     background: var(--pill-bg);
1055 |     border: 1px solid var(--pill-border);
1056 |     border-radius: 20px;
1057 |     padding: 2px 7px;
1058 |   }
1059 |
1060 |   .search-result-snippet {
1061 |     font-size: 12.5px;
1062 |     color: var(--text-dim);
1063 |     line-height: 1.5;
1064 |   }
1065 |
1066 |   .search-result-path {
1067 |     font-size: 11px;
1068 |     color: var(--text-faint);
1069 |     margin-top: 4px;
1070 |   }
1071 |
1072 |   mark.result-hit, mark.search-hit {
1073 |     background: var(--highlight-bg);
1074 |     color: var(--highlight-text);
1075 |     border-radius: 3px;
1076 |     padding: 0 3px;
1077 |   }
1078 |
1079 |   /* Archive / Trash rows */
1080 |   .archive-row {
1081 |     display: flex;
1082 |     align-items: center;
1083 |     gap: 12px;
1084 |     padding: 14px 16px;
1085 |     border-radius: 16px;
1086 |     background: var(--card-bg);
1087 |     border: 1px solid var(--card-border);
1088 |   }
1089 |
1090 |   .archive-row-title {
1091 |     flex: 1;
1092 |     font-weight: 600;
1093 |     font-size: 14.5px;
1094 |     color: var(--text);
1095 |   }
1096 |
1097 |   .archive-row-title span {
1098 |     display: block;
1099 |     font-size: 11.5px;
1100 |     color: var(--text-faint);
```

src/index.css (continued from previous page)

```
    font-weight: 400;
    margin-top: 2px;
  }

  .pinned-tag {
    display: inline-block;
    margin-left: 6px;
    font-size: 9.5px;
    font-weight: 800;
    color: var(--pin-active);
    background: rgba(255,196,84,0.15);
    border: 1px solid var(--pin-border);
    border-radius: 10px;
    padding: 1px 6px;
  }

  .archive-mini-btn {
    width: 34px;
    height: 34px;
    border-radius: 10px;
    border: none;
    background: var(--icon-bg);
    color: var(--text-dim);
    display: flex;
    align-items: center;
    justify-content: center;
    cursor: pointer;
  }

  .archive-mini-btn:hover {
    background: var(--icon-bg-hover);
    color: var(--text);
  }

  .archive-mini-btn.danger:hover {
    background: var(--danger-bg);
    color: var(--danger);
  }

  /* Toast */
  .toast {
    position: fixed;
    bottom: 28px;
    left: 50%;
    transform: translateX(-50%);
    background: var(--toast-bg);
    color: #fff;
    font-size: 13px;
    font-weight: 600;
    padding: 10px 18px;
    border-radius: 30px;
    border: 1px solid var(--divider);
    box-shadow: 0 10px 30px rgba(0,0,0,0.3);
    z-index: 60;
  }

  /* Code Export Modal & Code PDF Features */
  .code-pdf-btn {
    color: var(--accent1);
    position: relative;
  }

  .code-pdf-btn:hover {
    background: var(--pill-bg);
    color: var(--accent1);
  }

  .write-float-code {
    position: fixed;
```

src/index.css (continued from previous page)

```
    top: 18px;
    right: 68px;
    z-index: 20;
  }

  .write-float-pdf {
    position: fixed;
    top: 18px;
    right: 118px;
    z-index: 20;
  }

  .code-export-modal {
    width: 92vw;
    max-width: 980px;
    height: 85vh;
    max-height: 780px;
    background: var(--sheet-bg);
    border: 1px solid var(--sheet-border);
    border-radius: 20px;
    display: flex;
    flex-direction: column;
    overflow: hidden;
    box-shadow: 0 24px 60px rgba(0, 0, 0, 0.45), 0 0 0 1px rgba(255, 255, 255, 0.05);
    animation: modalSlideUp 0.22s cubic-bezier(0.16, 1, 0.3, 1);
  }

  @keyframes modalSlideUp {
    from {
      opacity: 0;
      transform: translateY(24px) scale(0.98);
    }
    to {
      opacity: 1;
      transform: translateY(0) scale(1);
    }
  }

  .code-export-header {
    padding: 18px 24px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    border-bottom: 1px solid var(--divider);
    background: rgba(0, 0, 0, 0.08);
  }

  .code-export-header-title {
    display: flex;
    align-items: center;
    gap: 14px;
  }

  .code-badge-icon {
    width: 40px;
    height: 40px;
    border-radius: 12px;
    background: var(--pill-bg);
    border: 1px solid var(--pill-border);
    color: var(--accent1);
    display: flex;
    align-items: center;
    justify-content: center;
  }

  .code-export-header-title h3 {
    margin: 0;
    font-size: 17px;
    font-weight: 700;
```

src/index.css (continued from previous page)

```
    color: var(--text);
    letter-spacing: -0.01em;
  }

  .code-export-header-title p {
    margin: 2px 0 0;
    font-size: 12px;
    color: var(--text-dim);
  }

  .code-export-header-actions {
    display: flex;
    align-items: center;
    gap: 12px;
  }

  .export-zip-btn {
    display: flex;
    align-items: center;
    gap: 8px;
    padding: 9px 16px;
    border-radius: 12px;
    border: 1px solid var(--pill-border);
    background: var(--pill-bg);
    color: var(--accent1);
    font-size: 13px;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.18s ease;
  }

  .export-zip-btn:hover {
    transform: translateY(-1px);
    filter: brightness(1.12);
    background: var(--card-bg-hover);
  }

  .export-pdf-main-btn {
    display: flex;
    align-items: center;
    gap: 8px;
    padding: 9px 18px;
    border-radius: 12px;
    border: none;
    background: linear-gradient(135deg, var(--accent1), var(--accent2));
    color: #fff;
    font-size: 13px;
    font-weight: 600;
    cursor: pointer;
    box-shadow: 0 4px 16px var(--accent-shadow);
    transition: all 0.18s ease;
  }

  .export-pdf-main-btn:hover {
    transform: translateY(-1px);
    box-shadow: 0 6px 22px var(--accent-shadow);
    filter: brightness(1.08);
  }

  .export-pdf-main-btn:disabled {
    opacity: 0.6;
    cursor: not-allowed;
    transform: none;
  }

  .code-export-body {
    flex: 1;
    display: flex;
    min-height: 0;
```

src/index.css (continued from previous page)

```
}

.code-file-list {
  width: 320px;
  border-right: 1px solid var(--divider);
  overflow-y: auto;
  padding: 10px;
  background: rgba(0, 0, 0, 0.04);
}

.code-list-title {
  font-size: 11px;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--text-faint);
  padding: 8px 12px 6px;
}

.code-file-item {
  width: 100%;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 8px 12px;
  margin-bottom: 4px;
  border-radius: 10px;
  border: 1px solid transparent;
  background: transparent;
  cursor: pointer;
  text-align: left;
  transition: all 0.15s ease;
}

.code-file-item:hover {
  background: var(--card-bg-hover);
  border-color: var(--card-border);
}

.code-file-item.active {
  background: var(--pill-bg);
  border-color: var(--pill-border);
}

.file-item-left {
  display: flex;
  align-items: center;
  gap: 10px;
  min-width: 0;
}

.file-icon {
  color: var(--text-faint);
  flex-shrink: 0;
}

.code-file-item.active .file-icon {
  color: var(--accent1);
}

.file-info {
  display: flex;
  flex-direction: column;
  min-width: 0;
}

.file-name {
  font-size: 13px;
  font-weight: 600;
```

src/index.css (continued from previous page)

```
1377 |     color: var(--text);
1378 |     white-space: nowrap;
1379 |     overflow: hidden;
1380 |     text-overflow: ellipsis;
1381 |   }
1382 |
1383 |   .file-path {
1384 |     font-size: 10.5px;
1385 |     color: var(--text-faint);
1386 |     white-space: nowrap;
1387 |     overflow: hidden;
1388 |     text-overflow: ellipsis;
1389 |   }
1390 |
1391 |   .file-meta {
1392 |     display: flex;
1393 |     flex-direction: column;
1394 |     align-items: flex-end;
1395 |     gap: 3px;
1396 |     flex-shrink: 0;
1397 |     margin-left: 8px;
1398 |   }
1399 |
1400 |   .file-badge {
1401 |     font-size: 9px;
1402 |     font-weight: 700;
1403 |     text-transform: uppercase;
1404 |     letter-spacing: 0.03em;
1405 |     padding: 1px 5px;
1406 |     border-radius: 4px;
1407 |     background: rgba(255, 255, 255, 0.08);
1408 |     color: var(--text-dim);
1409 |   }
1410 |
1411 |   .code-file-item.active .file-badge {
1412 |     background: var(--pill-border);
1413 |     color: var(--text);
1414 |   }
1415 |
1416 |   .file-lines {
1417 |     font-size: 10px;
1418 |     color: var(--text-faint);
1419 |   }
1420 |
1421 |   .code-preview-pane {
1422 |     flex: 1;
1423 |     display: flex;
1424 |     flex-direction: column;
1425 |     min-width: 0;
1426 |     background: rgba(0, 0, 0, 0.12);
1427 |   }
1428 |
1429 |   .code-pane-toolbar {
1430 |     padding: 10px 18px;
1431 |     border-bottom: 1px solid var(--divider);
1432 |     display: flex;
1433 |     align-items: center;
1434 |     justify-content: space-between;
1435 |     gap: 14px;
1436 |     background: rgba(0, 0, 0, 0.04);
1437 |   }
1438 |
1439 |   .pane-file-details {
1440 |     display: flex;
1441 |     flex-direction: column;
1442 |     min-width: 0;
1443 |   }
1444 |
1445 |   .pane-filename {
```

src/index.css (continued from previous page)

```
    font-family: monospace;
    font-size: 12.5px;
    font-weight: 700;
    color: var(--text);
  }

  .pane-desc {
    font-size: 11px;
    color: var(--text-dim);
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
  }

  .pane-actions {
    display: flex;
    align-items: center;
    gap: 8px;
    flex-shrink: 0;
  }

  .pane-btn {
    display: flex;
    align-items: center;
    gap: 6px;
    padding: 6px 12px;
    border-radius: 8px;
    border: 1px solid var(--card-border);
    background: var(--card-bg);
    color: var(--text);
    font-size: 12px;
    font-weight: 500;
    cursor: pointer;
    transition: all 0.15s ease;
  }

  .pane-btn:hover {
    background: var(--card-bg-hover);
    border-color: var(--divider);
  }

  .code-viewer-content {
    flex: 1;
    margin: 0;
    padding: 14px 18px;
    overflow: auto;
    font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, monospace;
    font-size: 11.5px;
    line-height: 1.6;
    color: var(--text);
    background: transparent;
    user-select: text;
  }

  .code-line {
    display: flex;
  }

  .code-line:hover {
    background: rgba(255, 255, 255, 0.03);
  }

  .code-line-number {
    display: inline-block;
    width: 44px;
    text-align: right;
    padding-right: 16px;
    color: var(--text-faint);
    opacity: 0.6;
```

src/index.css (continued from previous page)

```
    user-select: none;
    flex-shrink: 0;
  }

  .code-line-text {
    flex: 1;
    white-space: pre-wrap;
    word-break: break-all;
  }

  .code-export-footer {
    padding: 12px 22px;
    border-top: 1px solid var(--divider);
    display: flex;
    align-items: center;
    justify-content: space-between;
    background: rgba(0, 0, 0, 0.08);
  }

  .pdf-info-pill {
    display: flex;
    align-items: center;
    gap: 6px;
    font-size: 11.5px;
    color: var(--text-dim);
  }

  .direct-pdf-link {
    display: flex;
    align-items: center;
    gap: 5px;
    font-size: 12px;
    font-weight: 600;
    color: var(--accent1);
    text-decoration: none;
    padding: 5px 12px;
    border-radius: 8px;
    background: var(--pill-bg);
    border: 1px solid var(--pill-border);
    transition: all 0.15s ease;
  }

  .direct-pdf-link:hover {
    filter: brightness(1.15);
    text-decoration: underline;
  }

  @media (max-width: 768px) {
    .code-export-modal {
      width: 96vw;
      height: 92vh;
    }
    .code-export-body {
      flex-direction: column;
    }
    .code-file-list {
      width: 100%;
      height: 180px;
      border-right: none;
      border-bottom: 1px solid var(--divider);
    }
  }


  /* Constrain the existing controls at small viewports; retain their styling. */
  .card-drag-handle { touch-action: none; }
  .card-title, .card-body, .archive-row-title, .note-chip-label,
  .write-content, .float-panel-body-editable { overflow-wrap: anywhere; }
  .archive-row-title, .search-input { min-width: 0; }
```

src/index.css (continued from previous page)

```
  .float-panel-header { cursor: grab; touch-action: none; }
  .float-panel-header input, .float-panel-header button { touch-action: auto; }
  .float-panel-scroll-container { min-height: 0; }
  .bubble-settings-tray { min-height: 0; max-height: 65%; overflow-y: auto; }
  .float-panel-body-editable img { max-width: 100%; }
  .sheet { max-height: 95dvh; overflow-y: auto; }

  @media (max-width: 768px) {
    .code-export-header { flex-wrap: wrap; gap: 12px; padding: 14px 16px; flex-shrink: 0; }
    .code-export-header-title { min-width: 0; }
    .code-badge-icon { flex-shrink: 0; }
    .code-export-header-actions { flex-wrap: wrap; gap: 8px; }
    .code-export-footer { flex-wrap: wrap; gap: 10px; padding: 10px 14px; flex-shrink: 0; }
    .code-export-footer .footer-right { flex-wrap: wrap; }
    .code-file-list { flex-shrink: 0; height: min(180px, 20dvh); }
    .code-pane-toolbar { flex-wrap: wrap; gap: 8px; padding: 10px 12px; }
    .code-export-modal { height: 92dvh; }
  }
  @media (max-width: 480px) {
    .float-panel-header { flex-wrap: wrap; gap: 8px; padding: 10px 12px; }
    .float-panel-title-input { flex-basis: 100%; }
    .float-panel-actions { width: 100%; justify-content: flex-end; }
  }

```

src/utils/storage.ts

Persistence | 91 Lines

Role: Local storage engine for durable cross-session data storage

```
  // Keep the original key and record shape. A bad record must never be replaced
  // with an empty tree just because parsing or validation failed.
  export const STORAGE_KEY = 'pinboard-notes-data-v5';

  function validateData(data: any): any {
    const ids = new Set<string>();
    const validateNode = (node: any) => {
      if (!node || typeof node.id !== 'string' || !node.id || ids.has(node.id) ||
          typeof node.title !== 'string' || typeof node.content !== 'string' ||
          !Array.isArray(node.children)) throw new Error('Invalid saved note tree');
      ids.add(node.id);
      node.children.forEach(validateNode);
    };
    if (!data || !data.tree || data.tree.id !== 'root') throw new Error('Invalid saved root');
    validateNode(data.tree);
    if (data.trash !== undefined && !Array.isArray(data.trash)) throw new Error('Invalid saved recycle bin');
    const trashIds = new Set<string>();
    for (const entry of data.trash || []) {
      if (!entry || typeof entry.id !== 'string' || trashIds.has(entry.id) ||
          typeof entry.parentId !== 'string' || typeof entry.deletedAt !== 'number') {
        throw new Error('Invalid saved recycle bin entry');
      }
      trashIds.add(entry.id);
      validateNode(entry.node);
    }
    if (data.theme !== undefined && data.theme !== 'dark' && data.theme !== 'light') throw new Error('Invalid
> | saved theme');
    if (data.bubbleSettings !== undefined && (!data.bubbleSettings || typeof data.bubbleSettings !== 'object'))
> | {
      throw new Error('Invalid saved bubble settings');
    }
    const limits: Record<string, [number, number]> = {
      transparency: [0.2, 1], bubblePopSize: [48, 96], expansionWidth: [300, 720],
      expansionHeight: [260, 650], cropTop: [0, 200], cropBottom: [0, 200], cropHeight: [100, 400],
    };
    for (const [key, [min, max]] of Object.entries(limits)) {
      const value = data.bubbleSettings?.[key];
      if (value !== undefined && (typeof value !== 'number' || !Number.isFinite(value) || value < min || value
> | > max)) {
        throw new Error('Invalid saved bubble setting');
      }
    }
    return data;
  }

  export async function loadAppData(): Promise<any | null> {
    // The synchronous browser copy contains the most recent edits even if the
    // optional asynchronous host storage was still writing when a page closed.
    let local: string | null = null;
    try { local = window.localStorage.getItem(STORAGE_KEY); }
    catch { /* Try the original host storage if browser storage is unavailable. */ }
    if (local !== null) return validateData(JSON.parse(local));

    const storage = (window as any).storage;
    if (storage && typeof storage.get === 'function') {
      const result = await storage.get(STORAGE_KEY, false);
      if (result?.value) return validateData(JSON.parse(result.value));
    }
    return null;
  }

  let pendingRemote: string | null = null;
  let remoteWrite: Promise<boolean> | null = null;

  export function saveAppData(data: any): Promise<boolean> {
```

src/utils/storage.ts (continued from previous page)

```
    const serialized = JSON.stringify(data);
    let localSaved = false;
    try {
      window.localStorage.setItem(STORAGE_KEY, serialized);
      localSaved = true;
    } catch (error) {
      console.error('localStorage save failed:', error);
    }

    const storage = (window as any).storage;
    if (!storage || typeof storage.set !== 'function') return Promise.resolve(localSaved);
    // Serialize host writes and coalesce pending edits so old writes cannot
    // finish after new ones and overwrite the latest record.
    pendingRemote = serialized;
    if (!remoteWrite) {
      remoteWrite = (async () => {
        let saved = false;
        while (pendingRemote !== null) {
          const next = pendingRemote;
          pendingRemote = null;
          try { await storage.set(STORAGE_KEY, next, false); saved = true; }
          catch (error) { saved = false; console.warn('window.storage.set failed:', error); }
        }
        return saved;
      })().finally(() => { remoteWrite = null; });
    }
    return localSaved ? Promise.resolve(true) : remoteWrite;
  }

```

src/utils/tree.ts

Tree Algorithms | 237 Lines

Role: Tree hierarchy manipulation, sorting, pathing, and chip sync

```ts
import { NoteNode, SearchResult } from '../types';

export const generateUid = () =>
  Math.random().toString(36).slice(2, 9) + Date.now().toString(36);

export function createRootNode(): NoteNode {
  return {
    id: 'root',
    title: 'Home',
    content: '',
    archived: false,
    pinned: false,
    pinnedAt: null,
    order: 0,
    children: [],
    createdAt: Date.now(),
    updatedAt: Date.now(),
  };
}

export function cloneTree(node: NoteNode): NoteNode {
  return JSON.parse(JSON.stringify(node));
}

export function findNode(node: NoteNode, id: string): NoteNode | null {
  if (node.id === id) return node;
  for (const child of node.children || []) {
    const found = findNode(child, id);
    if (found) return found;
  }
  return null;
}

export function findParentNode(node: NoteNode, id: string): NoteNode | null {
  for (const child of node.children || []) {
    if (child.id === id) return node;
    const found = findParentNode(child, id);
    if (found) return found;
  }
  return null;
}

export function findPathIds(node: NoteNode, targetId: string, path: string[] = []): string[] | null {
  const currentPath = [...path, node.id];
  if (node.id === targetId) return currentPath;
  for (const child of node.children || []) {
    const found = findPathIds(child, targetId, currentPath);
    if (found) return found;
  }
  return null;
}

export function deleteNodeById(node: NoteNode, id: string): void {
  if (!node.children) return;
  node.children = node.children.filter((c) => c.id !== id);
  node.children.forEach((c) => deleteNodeById(c, id));
}

export function stripHtml(html: string): string {
  if (!html) return '';
  const div = document.createElement('div');
  div.innerHTML = html;
  div.querySelectorAll('br').forEach((br) => br.replaceWith('\n'));
  div.querySelectorAll('p, div, h1, h2, h3, h4, h5, h6, li, blockquote').forEach((block) => {
    block.prepend('\n');
```

src/utils/tree.ts (continued from previous page)

```
      block.append('\n');
    });
    return (div.textContent || '').replace(/\n{3,}/g, '\n\n').trim();
  }

  export function escapeHtml(str: string): string {
    if (!str) return '';
    const div = document.createElement('div');
    div.textContent = str;
    return div.innerHTML;
  }

  export function makeChipHtml(id: string, title: string, pinned: boolean = true): string {
    const pinnedClass = pinned ? 'note-chip-pinned' : '';
    const chip = document.createElement('div');
    chip.className = `note-chip ${pinnedClass}`;
    chip.setAttribute('contenteditable', 'false');
    chip.setAttribute('data-note-id', id);
    chip.innerHTML = `<span class="note-chip-pin"></span><span
> class="note-chip-label">${escapeHtml(title || 'Untitled')}</span>`;
    return chip.outerHTML;
  }

  export function contentHasChip(html: string, id: string): boolean {
    if (!html) return false;
    const div = document.createElement('div');
    div.innerHTML = html;
    return Array.from(div.querySelectorAll('.note-chip')).some((chip) =>
> chip.getAttribute('data-note-id') === id);
  }

  export function refreshChipTitles(container: HTMLElement | null, tree: NoteNode | null): void {
    if (!container || !tree) return;
    const chips = container.querySelectorAll('.note-chip');
    chips.forEach((chip) => {
      const id = chip.getAttribute('data-note-id');
      const node = id ? findNode(tree, id) : null;
      const label = chip.querySelector('.note-chip-label');
      if (label) label.textContent = node ? node.title || 'Untitled' : 'Untitled';
      chip.classList.toggle('note-chip-pinned', !!(node && node.pinned));
      chip.classList.toggle('note-chip-missing', !node);
    });
  }

  // Sorts children into Pinned (sorted by pinnedAt descending) and Normal (sorted by order)
  export function getSortedChildren(children: NoteNode[]): { pinned: NoteNode[]; normal: NoteNode[]
> } {
    const activeChildren = (children || []).filter((c) => !c.archived);

    // Pinned section: higher pinnedAt (more recently pinned) comes FIRST
    const pinned = activeChildren
      .filter((c) => c.pinned)
      .sort((a, b) => {
        const aTime = a.pinnedAt || a.updatedAt || 0;
        const bTime = b.pinnedAt || b.updatedAt || 0;
        if (bTime !== aTime) return bTime - aTime;
        return (a.order ?? 0) - (b.order ?? 0);
      });

    // Normal section: ordered by `order` property
    const normal = activeChildren
      .filter((c) => !c.pinned)
      .sort((a, b) => (a.order ?? 0) - (b.order ?? 0));

    return { pinned, normal };
  }

  // Reorders items when dragged
  export function reorderNodeChildren(
```

src/utils/tree.ts (continued from previous page)

```
    parent: NoteNode,
    sourceId: string,
    targetId: string,
    isPinnedList: boolean
  ): void {
    if (!parent.children) return;

    const { pinned, normal } = getSortedChildren(parent.children);
    const list = isPinnedList ? [...pinned] : [...normal];

    const sourceIndex = list.findIndex((c) => c.id === sourceId);
    const targetIndex = list.findIndex((c) => c.id === targetId);

    if (sourceIndex === -1 || targetIndex === -1 || sourceIndex === targetIndex) return;

    const [movedItem] = list.splice(sourceIndex, 1);
    list.splice(targetIndex, 0, movedItem);

    if (isPinnedList) {
      // Re-assign pinnedAt timestamps so the new visual sequence is preserved
      const now = Date.now();
      list.forEach((item, index) => {
        // Top items get higher timestamps so pinnedAt descending retains the order
        item.pinnedAt = now - index * 1000;
      });
    } else {
      // Re-assign normal order indices
      list.forEach((item, index) => {
        item.order = index;
      });
    }
  }

  function buildSnippet(text: string, query: string, radius = 42): string {
    const idx = text.toLowerCase().indexOf(query.toLowerCase());
    if (idx === -1) return text.slice(0, 90);
    const start = Math.max(0, idx - radius);
    const end = Math.min(text.length, idx + query.length + radius);
    let snippet = text.slice(start, end).trim();
    if (start > 0) snippet = '…' + snippet;
    if (end < text.length) snippet = snippet + '…';
    return snippet;
  }

  export function collectSearchResults(
    node: NoteNode,
    query: string,
    path: string[],
    out: SearchResult[]
  ): void {
    const q = query.toLowerCase();
    const nextPath = [...path, node.id];
    if (node.id !== 'root') {
      const title = node.title || '';
      const plain = stripHtml(node.content);
      const titleMatch = title.toLowerCase().includes(q);
      const contentMatch = plain.toLowerCase().includes(q);
      if (titleMatch || contentMatch) {
        out.push({
          id: node.id,
          title: title || 'Untitled',
          path: nextPath,
          archived: node.archived,
          pinned: node.pinned,
          matchType: titleMatch ? 'title' : 'content',
          snippet: contentMatch ? buildSnippet(plain, query) : '',
        });
      }
    }
```

src/utils/tree.ts (continued from previous page)

```
    (node.children || []).forEach((c) => collectSearchResults(c, query, nextPath, out));
  }

  export function highlightFirstMatch(root: HTMLElement, query: string): HTMLElement | null {
    if (!root || !query) return null;
    const q = query.toLowerCase();
    const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT, null);
    let node: Node | null;
    while ((node = walker.nextNode())) {
      const text = node.nodeValue || '';
      const idx = text.toLowerCase().indexOf(q);
      if (idx !== -1) {
        try {
          const range = document.createRange();
          range.setStart(node, idx);
          range.setEnd(node, idx + query.length);
          const mark = document.createElement('mark');
          mark.className = 'search-hit';
          range.surroundContents(mark);
          return mark;
        } catch (e) {
          return null;
        }
      }
    }
    return null;
  }

  export function pathTitles(tree: NoteNode, path: string[]): string[] {
    return path
      .slice(1, -1)
      .map((id) => {
        const n = findNode(tree, id);
        return n ? n.title || 'Untitled' : 'Untitled';
      });
  }

```

src/utils/codeExport.ts

Code Export | 343 Lines

Role: In-browser codebase catalog and client-side PDF compiler

```
import { jsPDF } from 'jspdf';

import appTsx from '../App.tsx?raw';
import mainTsx from '../main.tsx?raw';
import typesTs from '../types.ts?raw';
import viteEnvDTs from '../vite-env.d.ts?raw';
import indexCss from '../index.css?raw';
import storageTs from './storage.ts?raw';
import treeTs from './tree.ts?raw';
import codeExportTs from './codeExport.ts?raw';
import topBarTsx from '../components/TopBar.tsx?raw';
import boardViewTsx from '../components/BoardView.tsx?raw';
import noteCardTsx from '../components/NoteCard.tsx?raw';
import editorViewTsx from '../components/EditorView.tsx?raw';
import bubblePinTsx from '../components/BubblePin.tsx?raw';
import actionSheetTsx from '../components/ActionSheet.tsx?raw';
import searchModalTsx from '../components/SearchModal.tsx?raw';
import archiveTrashViewTsx from '../components/ArchiveTrashView.tsx?raw';
import codeExportModalTsx from '../components/CodeExportModal.tsx?raw';

export interface SourceFile {
  path: string;
  category: string;
  description: string;
  content: string;
  lines: number;
}

export const CODEBASE_FILES: SourceFile[] = [
  {
    path: 'src/App.tsx',
    category: 'Core',
    description: 'Main application logic, state orchestration, route handling, and PDF export trigger',
    content: appTsx,
    lines: appTsx.split('\n').length,
  },
  {
    path: 'src/types.ts',
    category: 'Types',
    description: 'TypeScript interfaces for NoteNode, TrashEntry, BubbleSettings, and ViewMode',
    content: typesTs,
    lines: typesTs.split('\n').length,
  },
  {
    path: 'src/components/BubblePin.tsx',
    category: 'UI',
    description: 'Editable floating bubble pin with customizable opacity, pop size, and viewport cropping',
    content: bubblePinTsx,
    lines: bubblePinTsx.split('\n').length,
  },
  {
    path: 'src/components/BoardView.tsx',
    category: 'UI',
    description: 'Interactive board view separating pinned importance hierarchy and unpinned notes',
    content: boardViewTsx,
    lines: boardViewTsx.split('\n').length,
  },
  {
    path: 'src/components/NoteCard.tsx',
    category: 'UI',
    description: 'Draggable note card with hold-to-reorder, pin indicators, and long-press menu',
    content: noteCardTsx,
    lines: noteCardTsx.split('\n').length,
  },
  {
```

src/utils/codeExport.ts (continued from previous page)

```
      path: 'src/components/EditorView.tsx',
      category: 'UI',
      description: 'Writing mode with rich text formatting, nested note chips, and standout bubble
> | pop button',
      content: editorViewTsx,
      lines: editorViewTsx.split('\n').length,
    },
    {
      path: 'src/components/TopBar.tsx',
      category: 'UI',
      description: 'Header with breadcrumb navigation, search, theme toggler, and code export
> | trigger',
      content: topBarTsx,
      lines: topBarTsx.split('\n').length,
    },
    {
      path: 'src/components/ActionSheet.tsx',
      category: 'UI',
      description: 'Context menu for pin/unpin, clipboard copy, PDF export, archive, and deletion',
      content: actionSheetTsx,
      lines: actionSheetTsx.split('\n').length,
    },
    {
      path: 'src/components/ArchiveTrashView.tsx',
      category: 'UI',
      description: 'Recycle bin and archive manager with restoration to parent writing pages',
      content: archiveTrashViewTsx,
      lines: archiveTrashViewTsx.split('\n').length,
    },
    {
      path: 'src/components/SearchModal.tsx',
      category: 'UI',
      description: 'Instant full-text search across titles and contents with highlighted snippets',
      content: searchModalTsx,
      lines: searchModalTsx.split('\n').length,
    },
    {
      path: 'src/components/CodeExportModal.tsx',
      category: 'UI',
      description: 'In-app codebase explorer, file inspector, and downloadable PDF modal',
      content: codeExportModalTsx,
      lines: codeExportModalTsx.split('\n').length,
    },
    {
      path: 'src/utils/tree.ts',
      category: 'Utils',
      description: 'Recursive tree algorithms, chronological pinned sorting, chips, and node
> | cloning',
      content: treeTs,
      lines: treeTs.split('\n').length,
    },
    {
      path: 'src/utils/storage.ts',
      category: 'Utils',
      description: 'Local storage persistence manager for state durability across sessions',
      content: storageTs,
      lines: storageTs.split('\n').length,
    },
    {
      path: 'src/utils/codeExport.ts',
      category: 'Utils',
      description: 'Client-side code bundle and in-browser PDF generator engine',
      content: codeExportTs,
      lines: codeExportTs.split('\n').length,
    },
    {
      path: 'src/main.tsx',
      category: 'Entry',
      description: 'Vite React application entry point and DOM root renderer',
```

src/utils/codeExport.ts (continued from previous page)

```
      content: mainTsx,
      lines: mainTsx.split('\n').length,
    },
    {
      path: 'src/vite-env.d.ts',
      category: 'Types',
      description: 'Vite client environment and module declarations for raw text assets',
      content: viteEnvDTs,
      lines: viteEnvDTs.split('\n').length,
    },
    {
      path: 'src/index.css',
      category: 'Styles',
      description: 'Complete CSS variables, dual dark/light themes, animations, and modal styles',
      content: indexCss,
      lines: indexCss.split('\n').length,
    },
  ];

  export const CORE_FEATURES_LIST = [
    {
      id: 'F01',
      title: 'Hierarchical Tree Architecture & Path Navigation',
      desc: 'Recursive note trees with deep paths, immutable mutations, and automated localStorage
> |persistence.',
    },
    {
      id: 'F02',
      title: 'Hierarchy of Importance: Pinning & Chronological Ordering',
      desc: 'Pinned notes elevate to the top in descending order of pinnedAt timestamp. Unpinning
> |restores previous position.',
    },
    {
      id: 'F03',
      title: 'Hold & Arrange Notes\' Order (Manual Drag & Drop)',
      desc: 'Hold-and-drag interaction with grip handles, drop indicators, and persistent sequential
> |order storage.',
    },
    {
      id: 'F04',
      title: 'Editable Floating Bubble Pin & Customizable Viewport',
      desc: 'Detached floating note with live two-way sync, opacity slider (20%-100%), pop size
> |(48-96px), and viewport cropping.',
    },
    {
      id: 'F05',
      title: 'Unique Standout Bubble Pop Button',
      desc: 'Distinct vibrant gradient, ambient halo glow, spinning sparkle icon, and dedicated
> |tooltip.',
    },
    {
      id: 'F06',
      title: 'Recycle Bin & Pinned Note Restoration Flow',
      desc: 'Deleted notes retain parent lineage. Restoring re-embeds note chips and navigates to
> |the parent writing page.',
    },
    {
      id: 'F07',
      title: 'Archive System with Instant Restoration',
      desc: 'Soft archival hides completed items from active boards while keeping hierarchy intact
> |for one-click unarchiving.',
    },
    {
      id: 'F08',
      title: 'Instant Global Full-Text Search',
      desc: 'Recursively searches titles and rich text with highlighted query snippets and deep
> |stack breadcrumb navigation.',
    },
    {
```

src/utils/codeExport.ts (continued from previous page)

```
193 |       id: 'F09',
194 |       title: 'Dual PDF Export Engines (Note & Full Codebase)',
195 |       desc: 'Note-level PDF generation and complete 90+ page unabridged codebase PDF with line
    > | numbers and specifications.',
196 |     },
197 |     {
198 |       id: 'F10',
199 |       title: 'Design System & Responsive Dual Themes',
200 |       desc: 'Midnight Dark and Warm Parchment Light modes with semantic CSS variable tokens.',
201 |     },
202 |   ];
203 |
204 |   export function generateClientCodePdf(): void {
205 |     const doc = new jsPDF({ unit: 'pt', format: 'a4', orientation: 'portrait' });
206 |     const pageWidth = doc.internal.pageSize.getWidth();
207 |     const pageHeight = doc.internal.pageSize.getHeight();
208 |     const marginX = 36;
209 |     const marginTop = 48;
210 |     const marginBottom = 40;
211 |     const contentWidth = pageWidth - marginX * 2;
212 |
213 |     // Cover Page
214 |     doc.setFillColor(30, 58, 138);
215 |     doc.rect(0, 0, pageWidth, 16, 'F');
216 |     doc.setFillColor(59, 130, 246);
217 |     doc.rect(0, 16, pageWidth, 4, 'F');
218 |
219 |     doc.setFont('helvetica', 'bold');
220 |     doc.setFontSize(30);
221 |     doc.setTextColor(30, 58, 138);
222 |     doc.text('Pinboard Notes', marginX, 86);
223 |
224 |     doc.setFont('helvetica', 'normal');
225 |     doc.setFontSize(13);
226 |     doc.setTextColor(71, 85, 105);
227 |     doc.text('Complete Architectural Specification & Unabridged Source Code', marginX, 108);
228 |
229 |     doc.setDrawColor(203, 213, 225);
230 |     doc.setLineWidth(1.5);
231 |     doc.line(marginX, 124, pageWidth - marginX, 124);
232 |
233 |     // Render Features
234 |     let curY = 150;
235 |     doc.setFont('helvetica', 'bold');
236 |     doc.setFontSize(12);
237 |     doc.setTextColor(30, 58, 138);
238 |     doc.text('Feature Specifications & Modules', marginX, curY);
239 |     curY += 20;
240 |
241 |     CORE_FEATURES_LIST.forEach((feat) => {
242 |       doc.setFont('helvetica', 'bold');
243 |       doc.setFontSize(9);
244 |       doc.setTextColor(30, 58, 138);
245 |       doc.text(`[${feat.id}] ${feat.title}`, marginX + 8, curY);
246 |       curY += 12;
247 |
248 |       doc.setFont('helvetica', 'normal');
249 |       doc.setFontSize(8);
250 |       doc.setTextColor(71, 85, 105);
251 |       const splitDesc = doc.splitTextToSize(feat.desc, contentWidth - 16);
252 |       doc.text(splitDesc, marginX + 16, curY);
253 |       curY += splitDesc.length * 10 + 6;
254 |     });
255 |
256 |     drawFooter(doc, marginX, contentWidth, pageHeight);
257 |
258 |     // Render Files
259 |     CODEBASE_FILES.forEach((file) => {
260 |       doc.addPage();
```

src/utils/codeExport.ts (continued from previous page)

```
      let y = marginTop;

      doc.setFillColor(30, 58, 138);
      doc.roundedRect(marginX, y, contentWidth, 28, 4, 4, 'F');

      doc.setFont('courier', 'bold');
      doc.setFontSize(10.5);
      doc.setTextColor(255, 255, 255);
      doc.text(file.path, marginX + 12, y + 18);

      doc.setFont('helvetica', 'normal');
      doc.setFontSize(8.5);
      doc.setTextColor(219, 234, 254);
      doc.text(`${file.category} • ${file.lines} Lines`, marginX + contentWidth - 120, y + 18);

      y += 36;

      const lines = file.content.split('\n');
      doc.setFont('courier', 'normal');
      doc.setFontSize(7.2);
      const lineHeight = 10.5;

      for (let i = 0; i < lines.length; i++) {
        if (y + lineHeight > pageHeight - marginBottom) {
          drawFooter(doc, marginX, contentWidth, pageHeight);
          doc.addPage();
          y = marginTop;

          doc.setFillColor(241, 245, 249);
          doc.roundedRect(marginX, y, contentWidth, 18, 3, 3, 'F');
          doc.setFont('courier', 'bold');
          doc.setFontSize(8);
          doc.setTextColor(71, 85, 105);
          doc.text(`${file.path} (continued)`, marginX + 8, y + 12);
          y += 26;
        }

        const rawLine = lines[i];
        const lineNumStr = String(i + 1).padStart(4, ' ') + ' | ';

        if (i % 2 === 0) {
          doc.setFillColor(252, 253, 255);
          doc.rect(marginX, y - 7.5, contentWidth, lineHeight, 'F');
        }

        doc.setFont('courier', 'bold');
        doc.setTextColor(148, 163, 184);
        doc.text(lineNumStr, marginX + 4, y);

        doc.setFont('courier', 'normal');
        doc.setTextColor(30, 41, 59);
        const wrapped = doc.splitTextToSize(rawLine || ' ', contentWidth - 52);
        doc.text(wrapped[0], marginX + 44, y);
        y += lineHeight;
        for (const continuation of wrapped.slice(1)) {
          if (y + lineHeight > pageHeight - marginBottom) {
            drawFooter(doc, marginX, contentWidth, pageHeight);
            doc.addPage();
            y = marginTop;
          }
          doc.setFont('courier', 'normal');
          doc.setFontSize(7.2);
          doc.setTextColor(30, 41, 59);
          doc.text(continuation, marginX + 44, y);
          y += lineHeight;
        }
      }

      drawFooter(doc, marginX, contentWidth, pageHeight);
```

src/utils/codeExport.ts (continued from previous page)

```
330 |     });
331 |
332 |     doc.save('Pinboard_Notes_Codebase.pdf');
333 |   }
334 |
335 |   function drawFooter(doc: jsPDF, marginX: number, contentWidth: number, pageHeight: number) {
336 |     const pageNum = (doc.internal as any).getCurrentPageInfo().pageNumber;
337 |     doc.setFont('helvetica', 'normal');
338 |     doc.setFontSize(7.5);
339 |     doc.setTextColor(148, 163, 184);
340 |     doc.text('Pinboard Notes — Complete Codebase & Feature Specification', marginX, pageHeight -
    > | 18);
341 |     doc.text(`Page ${pageNum}`, marginX + contentWidth - 36, pageHeight - 18);
342 |   }
343 |
```

src/components/TopBar.tsx

UI Components | 113 Lines

Role: Navigation top bar, breadcrumb stack, and global tools

```tsx
import React from 'react';
import { ChevronLeft, X, Sun, Moon, Search, Trash2, Archive, FileCode } from 'lucide-react';
import { NoteNode, ViewMode, ThemeMode } from '../types';

interface TopBarProps {
  mode: ViewMode;
  currentNode: NoteNode;
  stack: string[];
  theme: ThemeMode;
  trashAreaNode: NoteNode | null;
  onSetMode: (mode: ViewMode) => void;
  onBack: () => void;
  onToggleTheme: () => void;
  onOpenSearch: () => void;
  onTitleChange: (newTitle: string) => void;
  onOpenTrashForArea: () => void;
  onOpenCodeExport: () => void;
}

export const TopBar: React.FC<TopBarProps> = ({
  mode,
  currentNode,
  stack,
  theme,
  trashAreaNode,
  onSetMode,
  onBack,
  onToggleTheme,
  onOpenSearch,
  onTitleChange,
  onOpenTrashForArea,
  onOpenCodeExport,
}) => {
  if (mode === 'write') return null; // Writing mode uses floating controls

  return (
    <div className="pb-topbar">
      <div className="pb-topbar-icons">
        <div className="pb-topbar-icons-left">
          {mode === 'archive' || mode === 'trash' ? (
            <button className="icon-btn" onClick={() => onSetMode('board')} title="Close view">
              <X size={18} />
            </button>
          ) : stack.length > 1 ? (
            <button className="icon-btn" onClick={onBack} title="Back">
              <ChevronLeft size={20} />
            </button>
          ) : (
            <div style={{ width: 40 }} />
          )}
        </div>

        <div className="pb-topbar-icons-right">
          <button
            className="icon-btn code-pdf-btn"
            onClick={onOpenCodeExport}
            title="Download Project Code as PDF"
          >
            <FileCode size={18} />
          </button>

          <button
            className="icon-btn"
            onClick={onToggleTheme}
            title={theme === 'dark' ? 'Switch to light theme' : 'Switch to dark theme'}
```

src/components/TopBar.tsx (continued from previous page)

```
            >
              {theme === 'dark' ? <Sun size={17} /> : <Moon size={17} />}
            </button>

            <button className="icon-btn" onClick={onOpenSearch} title="Search notes">
              <Search size={17} />
            </button>

            {mode === 'board' && (
              <>
                <button
                  className="icon-btn"
                  onClick={onOpenTrashForArea}
                  title="Recycle bin for this page"
                >
                  <Trash2 size={17} />
                </button>
                <button
                  className="icon-btn"
                  onClick={() => onSetMode('archive')}
                  title="Archive for this page"
                >
                  <Archive size={17} />
                </button>
              </>
            )}
          </div>
        </div>

        <div className="pb-title-row">
          <input
            className="pb-title-input"
            value={
              mode === 'archive'
                ? 'Archive'
                : mode === 'trash'
                ? `Recycle Bin — ${trashAreaNode ? trashAreaNode.title || 'Untitled' : 'Home'}`
                : currentNode.title
            }
            placeholder="Untitled Note"
            disabled={currentNode.id === 'root' || mode === 'archive' || mode === 'trash'}
            onChange={(e) => onTitleChange(e.target.value)}
          />
        </div>
      </div>
    );
  };

```

src/components/BoardView.tsx

UI Components | 129 Lines

Role: Interactive board separating pinned hierarchy and regular notes

```tsx
import React, { useState, useRef } from 'react';
import { Plus, Pin, FileText } from 'lucide-react';
import { NoteNode } from '../types';
import { NoteCard } from './NoteCard';
import { getSortedChildren, reorderNodeChildren, findNode } from '../utils/tree';

interface BoardViewProps {
  currentNode: NoteNode;
  onOpenNote: (id: string) => void;
  onLongPressNote: (id: string) => void;
  onAddChild: () => void;
  onUpdateTree: (mutator: (tree: NoteNode) => void) => void;
}

export const BoardView: React.FC<BoardViewProps> = ({
  currentNode,
  onOpenNote,
  onLongPressNote,
  onAddChild,
  onUpdateTree,
}) => {
  const [dragSourceId, setDragSourceId] = useState<string | null>(null);
  const [dragSourceIsPinned, setDragSourceIsPinned] = useState<boolean>(false);
  const lastDragTarget = useRef<string | null>(null);

  const { pinned, normal } = getSortedChildren(currentNode.children || []);
  const totalNotes = pinned.length + normal.length;

  const handleDragStartItem = (id: string, isPinned: boolean) => {
    setDragSourceId(id);
    setDragSourceIsPinned(isPinned);
    lastDragTarget.current = id;
  };

  const handleDragOverItem = (targetId: string, isPinned: boolean) => {
    if (!dragSourceId || dragSourceId === targetId) return;
    if (dragSourceIsPinned !== isPinned) return; // Prevent cross-section drag confusion
    if (lastDragTarget.current === targetId) return;
    lastDragTarget.current = targetId;

    onUpdateTree((tree) => {
      const activeParent = findNode(tree, currentNode.id);
      if (!activeParent) return;
      // Reorder in place
      reorderNodeChildren(activeParent, dragSourceId, targetId, isPinned);
    });
  };

  const handleDragEndItem = () => {
    setDragSourceId(null);
    lastDragTarget.current = null;
  };

  return (
    <>
      <div className="board-scroll">
        <div className="board-inner">
          {totalNotes === 0 ? (
            <div className="board-empty">
              <div className="empty-icon-circle">
                <FileText size={28} />
              </div>
              <p>No notes here yet</p>
              <span className="empty-sub">Tap the floating + button to create a new note</span>
            </div>
```

src/components/BoardView.tsx (continued from previous page)

```
            ) : (
              <>
                {/* Pinned Section - Takes Priority at the Top */}
                {pinned.length > 0 && (
                  <div className="section-block">
                    <div className="section-header">
                      <Pin size={14} className="pin-hdr-icon" />
                      <span>PINNED ({pinned.length})</span>
                      <div className="section-hdr-line" />
                    </div>
                    <div className="cards-grid">
                      {pinned.map((child) => (
                        <NoteCard
                          key={child.id}
                          node={child}
                          isPinnedSection={true}
                          onOpen={onOpenNote}
                          onLongPress={onLongPressNote}
                          onDragStartItem={handleDragStartItem}
                          onDragOverItem={handleDragOverItem}
                          onDragEndItem={handleDragEndItem}
                        />
                      ))}
                    </div>
                  </div>
                )}

                {/* Normal Notes Section */}
                {normal.length > 0 && (
                  <div className="section-block">
                    {pinned.length > 0 && (
                      <div className="section-header">
                        <span>OTHER NOTES ({normal.length})</span>
                        <div className="section-hdr-line" />
                      </div>
                    )}
                    <div className="cards-grid">
                      {normal.map((child) => (
                        <NoteCard
                          key={child.id}
                          node={child}
                          isPinnedSection={false}
                          onOpen={onOpenNote}
                          onLongPress={onLongPressNote}
                          onDragStartItem={handleDragStartItem}
                          onDragOverItem={handleDragOverItem}
                          onDragEndItem={handleDragEndItem}
                        />
                      ))}
                    </div>
                  </div>
                )}
              </>
            )}
          </div>
        </div>

        <button className="fab-add" onClick={onAddChild} title="Create Pin + (Add New Note)">
          <Plus size={24} />
        </button>
      </>
    );
  };

```

src/components/NoteCard.tsx

UI Components | 141 Lines

Role: Interactive card with hold-to-reorder and long-press menu

```tsx
import React, { useRef, useState, useEffect } from 'react';
import { GripVertical, Pin } from 'lucide-react';
import { NoteNode } from '../types';
import { stripHtml } from '../utils/tree';

interface NoteCardProps {
  node: NoteNode;
  isPinnedSection: boolean;
  onOpen: (id: string) => void;
  onLongPress: (id: string) => void;
  onDragStartItem: (id: string, isPinned: boolean) => void;
  onDragOverItem: (id: string, isPinned: boolean) => void;
  onDragEndItem: () => void;
}

export const NoteCard: React.FC<NoteCardProps> = ({
  node,
  isPinnedSection,
  onOpen,
  onLongPress,
  onDragStartItem,
  onDragOverItem,
  onDragEndItem,
}) => {
  const pressTimer = useRef<any>(null);
  const longFired = useRef(false);
  const [isDragging, setIsDragging] = useState(false);
  const pressStart = useRef({ x: 0, y: 0 });
  const pointerDrag = useRef(false);

  useEffect(() => () => clearTimeout(pressTimer.current), []);

  const childCount = (node.children || []).filter((c) => !c.archived).length;
  const excerpt = stripHtml(node.content).slice(0, 90);

  const startTimer = (e: React.PointerEvent) => {
    if (e.button !== 0) return;
    clearTimer();
    pressStart.current = { x: e.clientX, y: e.clientY };
    longFired.current = false;
    pressTimer.current = setTimeout(() => {
      longFired.current = true;
      onLongPress(node.id);
    }, 450);
  };

  const clearTimer = () => {
    if (pressTimer.current) clearTimeout(pressTimer.current);
  };

  const handleClick = (e: React.MouseEvent) => {
    if (longFired.current) {
      longFired.current = false;
      return;
    }
    onOpen(node.id);
  };

  const handleDragStart = (e: React.DragEvent) => {
    clearTimer();
    longFired.current = true;
    setIsDragging(true);
    e.dataTransfer.effectAllowed = 'move';
    e.dataTransfer.setData('text/plain', node.id);
    onDragStartItem(node.id, isPinnedSection);
```

src/components/NoteCard.tsx (continued from previous page)

```
    };

    const handleDragOver = (e: React.DragEvent) => {
      e.preventDefault();
      e.dataTransfer.dropEffect = 'move';
      onDragOverItem(node.id, isPinnedSection);
    };

    const handleDragEnd = () => {
      setIsDragging(false);
      onDragEndItem();
    };

    return (
      <div
        draggable
        onDragStart={handleDragStart}
        onDragOver={handleDragOver}
        onDragEnd={handleDragEnd}
        className={`note-card ${node.pinned ? 'pinned-glow' : ''} ${isDragging ? 'dragging' : ''}`}
        data-note-id={node.id}
        data-pinned-section={isPinnedSection}
        onPointerDown={startTimer}
        onPointerMove={(e) => {
          if (Math.abs(e.clientX - pressStart.current.x) > 8 || Math.abs(e.clientY -
> pressStart.current.y) > 8) clearTimer();
        }}
        onPointerUp={clearTimer}
        onPointerLeave={clearTimer}
        onPointerCancel={clearTimer}
        onClick={handleClick}
      >
        <div className="card-drag-handle" title="Hold and drag to reorder note"
          onPointerDown={(e) => {
            if (e.button !== 0) return;
            e.stopPropagation();
            e.preventDefault();
            clearTimer();
            pointerDrag.current = true;
            longFired.current = true;
            setIsDragging(true);
            e.currentTarget.setPointerCapture(e.pointerId);
            onDragStartItem(node.id, isPinnedSection);
          }}
          onPointerMove={(e) => {
            if (!pointerDrag.current) return;
            e.stopPropagation();
            const target = document.elementFromPoint(e.clientX,
> e.clientY)?.closest<HTMLElement>('.note-card');
            if (target?.dataset.noteId) onDragOverItem(target.dataset.noteId,
> target.dataset.pinnedSection === 'true');
          }}
          onPointerUp={(e) => {
            e.stopPropagation();
            if (pointerDrag.current) { pointerDrag.current = false; handleDragEnd(); }
          }}
          onPointerCancel={() => { pointerDrag.current = false; handleDragEnd(); }}
          onClick={(e) => { e.stopPropagation(); longFired.current = false; }}
        >
          <GripVertical size={14} />
        </div>

        <span className={`pin-emoji ${node.pinned ? 'pin-emoji-active' : ''}`}>
                  Ø=ÜÌ
        </span>

        {childCount > 0 && <div className="count-badge">{childCount}</div>}

        <div className="card-pill">
```

src/components/NoteCard.tsx (continued from previous page)

```
132 |           {node.pinned ? 'PINNED NOTE' : 'NOTE'}
133 |         </div>
134 |
135 |         <div className="card-title">{node.title || 'Untitled'}</div>
136 |         <div className="card-divider" />
137 |         <div className="card-body">{excerpt || 'No written notes yet.'}</div>
138 |       </div>
139 |     );
140 |   };
141 |
```

src/components/EditorView.tsx

UI Components | 314 Lines

Role: Rich text writing canvas, formatting toolbar, and nested chips

```tsx
import React, { useRef, useState, useEffect, useCallback } from 'react';
import {
  ChevronLeft,
  LayoutGrid,
  Bold,
  Italic,
  Underline,
  List,
  ListOrdered,
  Image as ImageIcon,
  Pin,
  Sparkles,
  Download,
  FileCode,
} from 'lucide-react';
import { NoteNode } from '../types';
import { makeChipHtml, refreshChipTitles, findNode } from '../utils/tree';

interface EditorViewProps {
  currentNode: NoteNode;
  tree: NoteNode;
  stack: string[];
  onBack: () => void;
  onSetModeBoard: () => void;
  onTitleChange: (newTitle: string) => void;
  onContentChange: (newHtml: string) => void;
  onInsertNestedNote: () => void;
  onPopOutBubble: () => void;
  onNavigateToNote: (id: string) => void;
  onLongPressNote: (id: string) => void;
  onShowToast: (msg: string) => void;
  onOpenCodeExport: () => void;
  onDownloadCurrentNotePdf: () => void;
}

export const EditorView: React.FC<EditorViewProps> = ({
  currentNode,
  tree,
  stack,
  onBack,
  onSetModeBoard,
  onTitleChange,
  onContentChange,
  onInsertNestedNote,
  onPopOutBubble,
  onNavigateToNote,
  onLongPressNote,
  onShowToast,
  onOpenCodeExport,
  onDownloadCurrentNotePdf,
}) => {
  const [editorFocused, setEditorFocused] = useState(false);
  const editorRef = useRef<HTMLDivElement>(null);
  const writeScrollRef = useRef<HTMLDivElement>(null);

  const chipPressTimer = useRef<any>(null);
  const chipLongFired = useRef(false);
  const displayedNoteId = useRef<string | null>(null);

  // Sync content into DOM editor element when current note changes
  useEffect(() => {
    if (editorRef.current && currentNode) {
      if (displayedNoteId.current !== currentNode.id || editorRef.current.innerHTML !== (currentNode.content || '')) {
        editorRef.current.innerHTML = currentNode.content || '';
```

src/components/EditorView.tsx (continued from previous page)

```
65 |         }
66 |         displayedNoteId.current = currentNode.id;
67 |         refreshChipTitles(editorRef.current, tree);
68 |       }
69 |     }, [currentNode.id, currentNode.content]);
70 |
71 |     useEffect(() => () => clearTimeout(chipPressTimer.current), []);
72 |
73 |     // Keep chip titles and pin states in sync with tree updates
74 |     useEffect(() => {
75 |       if (editorRef.current && tree) {
76 |         refreshChipTitles(editorRef.current, tree);
77 |       }
78 |     }, [tree]);
79 |
80 |     const scrollCaretIntoView = useCallback(() => {
81 |       const scroller = writeScrollRef.current;
82 |       if (!scroller) return;
83 |       const sel = window.getSelection();
84 |       if (!sel || sel.rangeCount === 0) return;
85 |       const range = sel.getRangeAt(0).cloneRange();
86 |       range.collapse(true);
87 |       let rect = range.getClientRects()[0];
88 |       if (!rect) {
89 |         const marker = document.createElement('span');
90 |         marker.appendChild(document.createTextNode('\u200b'));
91 |         range.insertNode(marker);
92 |         rect = marker.getBoundingClientRect();
93 |         marker.parentNode?.removeChild(marker);
94 |       }
95 |       if (!rect || (rect.top === 0 && rect.bottom === 0)) return;
96 |
97 |       const toolbarBuffer = 96;
98 |       const topBuffer = 84;
99 |       const viewportBottom = window.innerHeight - toolbarBuffer;
100 |      if (rect.bottom > viewportBottom) {
101 |        scroller.scrollTop += rect.bottom - viewportBottom + 24;
102 |      } else if (rect.top < topBuffer) {
103 |        scroller.scrollTop -= topBuffer - rect.top + 24;
104 |      }
105 |    }, []);
106 |
107 |    const handleEditorInput = () => {
108 |      if (!editorRef.current) return;
109 |      const html = editorRef.current.innerHTML;
110 |      onContentChange(html);
111 |      requestAnimationFrame(scrollCaretIntoView);
112 |    };
113 |
114 |    const exec = (cmd: string, value: string | null = null) => {
115 |      if (editorRef.current) editorRef.current.focus();
116 |      document.execCommand(cmd, false, value as any);
117 |      handleEditorInput();
118 |    };
119 |
120 |    const handleChipPointerDown = (e: React.PointerEvent) => {
121 |      const chip = (e.target as HTMLElement).closest?.('.note-chip');
122 |      if (!chip) return;
123 |      const id = chip.getAttribute('data-note-id');
124 |      if (!id) return;
125 |
126 |      chipLongFired.current = false;
127 |      chipPressTimer.current = setTimeout(() => {
128 |        chipLongFired.current = true;
129 |        onLongPressNote(id);
130 |      }, 450);
131 |    };
132 |
133 |    const handleChipPointerClear = () => {
```

src/components/EditorView.tsx (continued from previous page)

```
    if (chipPressTimer.current) clearTimeout(chipPressTimer.current);
  };

  const handleEditorClick = (e: React.MouseEvent) => {
    const chip = (e.target as HTMLElement).closest?.('.note-chip');
    if (chip) {
      e.preventDefault();
      e.stopPropagation();
      if (chipLongFired.current) {
        chipLongFired.current = false;
        return;
      }
      const id = chip.getAttribute('data-note-id');
      if (id) {
        if (!findNode(tree, id)) {
          onShowToast('This nested note was moved or deleted');
          return;
        }
        onNavigateToNote(id);
      }
    }
  };

  const focusEditorAnywhere = (e: React.MouseEvent) => {
    if (!editorRef.current) return;
    if (editorRef.current.contains(e.target as Node)) return;
    editorRef.current.focus();
    const range = document.createRange();
    range.selectNodeContents(editorRef.current);
    range.collapse(false);
    const sel = window.getSelection();
    sel?.removeAllRanges();
    sel?.addRange(range);
  };

  const handleImportImage = () => {
    let url: string | null = null;
    try {
      url = window.prompt('Paste image URL (e.g. https://... or data:image...)');
    } catch (e) {
      url = null;
    }
    if (url) exec('insertImage', url);
  };

  return (
    <div className="write-scroll" ref={writeScrollRef} onClick={focusEditorAnywhere}>
      {stack.length > 1 && (
        <button
          className="icon-btn write-float-back"
          onClick={(e) => {
            e.stopPropagation();
            onBack();
          }}
          title="Back to parent note page"
        >
          <ChevronLeft size={20} />
        </button>
      )}

      {/* Floating Header Actions in Writing Mode */}
      <button
        className="icon-btn write-float-code"
        onClick={(e) => {
          e.stopPropagation();
          onOpenCodeExport();
        }}
        title="Download Project Codebase as PDF"
      >
```

src/components/EditorView.tsx (continued from previous page)

```
203 |           <FileCode size={18} />
204 |         </button>
205 |
206 |         <button
207 |           className="icon-btn write-float-pdf"
208 |           onClick={(e) => {
209 |             e.stopPropagation();
210 |             onDownloadCurrentNotePdf();
211 |           }}
212 |           title="Download this Note as PDF"
213 |         >
214 |           <Download size={18} />
215 |         </button>
216 |
217 |         <button
218 |           className="icon-btn write-float-grid"
219 |           onClick={(e) => {
220 |             e.stopPropagation();
221 |             onSetModeBoard();
222 |           }}
223 |           title="View grid of nested notes inside this page"
224 |         >
225 |           <LayoutGrid size={18} />
226 |         </button>
227 |
228 |         <div className={`write-page ${editorFocused ? 'glow-focus' : ''}`}>
229 |           <input
230 |             className="write-title-input"
231 |             value={currentNode.title}
232 |             placeholder="Untitled Note"
233 |             disabled={currentNode.id === 'root'}
234 |             onChange={(e) => onTitleChange(e.target.value)}
235 |             onClick={(e) => e.stopPropagation()}
236 |           />
237 |
238 |           <div
239 |             ref={editorRef}
240 |             className="write-content"
241 |             contentEditable
242 |             suppressContentEditableWarning
243 |             onInput={handleEditorInput}
244 |             onClick={handleEditorClick}
245 |             onPointerDown={handleChipPointerDown}
246 |             onPointerUp={handleChipPointerClear}
247 |             onPointerLeave={handleChipPointerClear}
248 |             onPointerCancel={handleChipPointerClear}
249 |             onFocus={() => setEditorFocused(true)}
250 |             onBlur={() => setEditorFocused(false)}
251 |           />
252 |         </div>
253 |
254 |         {/* Floating Formatting Toolbar with Unique Bubble Pop Button */}
255 |         <div className="write-toolbar" onClick={(e) => e.stopPropagation()} onMouseDown={(e) =>
    > |e.preventDefault()}>
256 |           <button className="tb-btn" onClick={() => exec('formatBlock', 'H1')}>
257 |             <span style={{ fontWeight: 800, fontSize: 13 }}>H1</span>
258 |           </button>
259 |           <button className="tb-btn" onClick={() => exec('formatBlock', 'H2')}>
260 |             <span style={{ fontWeight: 700, fontSize: 13 }}>H2</span>
261 |           </button>
262 |
263 |           <div className="tb-sep" />
264 |
265 |           <button className="tb-btn" onClick={() => exec('bold')} title="Bold">
266 |             <Bold size={16} />
267 |           </button>
268 |           <button className="tb-btn" onClick={() => exec('italic')} title="Italic">
269 |             <Italic size={16} />
270 |           </button>
```

src/components/EditorView.tsx (continued from previous page)

```
271 |           <button className="tb-btn" onClick={() => exec('underline')} title="Underline">
272 |             <Underline size={16} />
273 |           </button>
274 |
275 |           <div className="tb-sep" />
276 |
277 |           <button className="tb-btn" onClick={() => exec('insertUnorderedList')} title="Bullet
    > |List">
278 |             <List size={16} />
279 |           </button>
280 |           <button className="tb-btn" onClick={() => exec('insertOrderedList')} title="Numbered
    > |List">
281 |             <ListOrdered size={16} />
282 |           </button>
283 |
284 |           <div className="tb-sep" />
285 |
286 |           <button className="tb-btn" onClick={handleImportImage} title="Insert Image">
287 |             <ImageIcon size={16} />
288 |           </button>
289 |
290 |           <div className="tb-sep" />
291 |
292 |           {/* Pin + Button */}
293 |           <button
294 |             className="tb-btn accent-pin"
295 |             onClick={onInsertNestedNote}
296 |             title="Insert a nested pinned note chip here"
297 |           >
298 |             <Pin size={16} />
299 |           </button>
300 |
301 |           {/* Unique Standout Bubble Pop Button */}
302 |           <button
303 |             className="tb-btn unique-bubble-pop"
304 |             onClick={onPopOutBubble}
305 |             title="Pop out this note into an editable floating bubble pin!"
306 |           >
307 |             <Sparkles size={16} className="sparkle-spin" />
308 |             <span className="bubble-pop-text">Bubble Pin</span>
309 |           </button>
310 |         </div>
311 |       </div>
312 |     );
313 |   };
314 |
```

src/components/BubblePin.tsx

UI Components | 467 Lines

Role: Detached editable floating bubble pin with viewport cropping

```tsx
import React, { useState, useRef, useEffect } from 'react';
import {
  X,
  Copy,
  ExternalLink,
  Minimize2,
  Settings,
  Crop,
  Check,
  RotateCcw,
} from 'lucide-react';
import { NoteNode, BubbleSettings } from '../types';
import { stripHtml } from '../utils/tree';

interface BubblePinProps {
  floatingNode: NoteNode | null;
  settings: BubbleSettings;
  onUpdateSettings: (newSettings: Partial<BubbleSettings>) => void;
  onUpdateNodeContent: (id: string, title: string, content: string) => void;
  onCopyNote: (node: NoteNode) => void;
  onShowToast: (msg: string) => void;
  onClose: () => void;
}

export const DEFAULT_BUBBLE_SETTINGS: BubbleSettings = {
  transparency: 0.92,
  bubblePopSize: 64,
  expansionWidth: 460,
  expansionHeight: 420,
  cropEnabled: false,
  cropTop: 0,
  cropBottom: 0,
  cropHeight: 250,
};

export const BubblePin: React.FC<BubblePinProps> = ({
  floatingNode,
  settings,
  onUpdateSettings,
  onUpdateNodeContent,
  onCopyNote,
  onShowToast,
  onClose,
}) => {
  const [expanded, setExpanded] = useState(false);
  const [showSettings, setShowSettings] = useState(false);
  const [pos, setPos] = useState({ x: window.innerWidth - 90, y: window.innerHeight - 200 });
  const [panelPos, setPanelPos] = useState<{ x: number; y: number } | null>(null);
  const panelRef = useRef<HTMLDivElement>(null);
  const latestNode = useRef(floatingNode);
  latestNode.current = floatingNode;

  const bubbleDrag = useRef({
    dragging: false,
    moved: false,
    startX: 0,
    startY: 0,
    origX: 0,
    origY: 0,
    panel: false,
  });

  const editorRef = useRef<HTMLDivElement>(null);
  const pipWindowRef = useRef<any>(null);
  const pipBodyRef = useRef<HTMLElement | null>(null);
```

src/components/BubblePin.tsx (continued from previous page)

```
  66 |
  67 |     useEffect(() => {
  68 |       setExpanded(false);
  69 |       setShowSettings(false);
  70 |       pipWindowRef.current?.close();
  71 |       pipWindowRef.current = null;
  72 |       pipBodyRef.current = null;
  73 |     }, [floatingNode?.id]);
  74 |
  75 |     useEffect(() => {
  76 |       const clamp = () => {
  77 |         setPos((p) => ({
  78 |           x: Math.max(8, Math.min(p.x, window.innerWidth - settings.bubblePopSize - 8)),
  79 |           y: Math.max(8, Math.min(p.y, window.innerHeight - settings.bubblePopSize - 8)),
  80 |         }));
  81 |         const rect = panelRef.current?.getBoundingClientRect();
  82 |         if (rect) setPanelPos((p) => p && ({
  83 |           x: Math.max(8, Math.min(p.x, window.innerWidth - rect.width - 8)),
  84 |           y: Math.max(8, Math.min(p.y, window.innerHeight - rect.height - 8)),
  85 |         }));
  86 |       };
  87 |       clamp();
  88 |       window.addEventListener('resize', clamp);
  89 |       return () => window.removeEventListener('resize', clamp);
  90 |     }, [settings.bubblePopSize, settings.expansionWidth, settings.expansionHeight, expanded,
  > | showSettings]);
  91 |
  92 |     useEffect(() => {
  93 |       return () => {
  94 |         if (pipWindowRef.current) {
  95 |           try {
  96 |             pipWindowRef.current.close();
  97 |           } catch (e) {}
  98 |         }
  99 |       };
 100 |     }, []);
 101 |
 102 |     // Sync content when floating node changes or opens
 103 |     useEffect(() => {
 104 |       if (expanded && editorRef.current && floatingNode) {
 105 |         if (editorRef.current.innerHTML !== (floatingNode.content || '')) {
 106 |           editorRef.current.innerHTML = floatingNode.content || '';
 107 |         }
 108 |       }
 109 |       if (floatingNode && pipWindowRef.current && pipBodyRef.current) {
 110 |         const h = pipWindowRef.current.document.querySelector('h2');
 111 |         if (h) h.textContent = floatingNode.title || 'Untitled';
 112 |         if (pipBodyRef.current.innerHTML !== (floatingNode.content || ''))
  > | pipBodyRef.current.innerHTML = floatingNode.content || '';
 113 |       }
 114 |     }, [expanded, floatingNode]);
 115 |
 116 |     useEffect(() => {
 117 |       if (settings.cropEnabled && editorRef.current?.parentElement) {
 118 |         editorRef.current.parentElement.scrollTop = settings.cropTop;
 119 |       }
 120 |     }, [settings.cropEnabled, settings.cropTop, expanded, floatingNode?.id]);
 121 |
 122 |     if (!floatingNode) return null;
 123 |
 124 |     const handlePointerDown = (e: React.PointerEvent) => {
 125 |       const panel = e.currentTarget.classList.contains('float-panel-header');
 126 |       if (panel && (e.target as HTMLElement).closest('input, button')) return;
 127 |       const rect = panelRef.current?.getBoundingClientRect();
 128 |       bubbleDrag.current = {
 129 |         dragging: true,
 130 |         moved: false,
 131 |         startX: e.clientX,
 132 |         startY: e.clientY,
```

```
        origX: panel && rect ? rect.left : pos.x,
        origY: panel && rect ? rect.top : pos.y,
        panel,
      };
      (e.currentTarget as HTMLElement).setPointerCapture?.(e.pointerId);
    };

    const handlePointerMove = (e: React.PointerEvent) => {
      const d = bubbleDrag.current;
      if (!d.dragging) return;
      const dx = e.clientX - d.startX;
      const dy = e.clientY - d.startY;
      if (Math.abs(dx) > 5 || Math.abs(dy) > 5) d.moved = true;

      const rect = panelRef.current?.getBoundingClientRect();
      const width = d.panel && rect ? rect.width : settings.bubblePopSize;
      const height = d.panel && rect ? rect.height : settings.bubblePopSize;
      const nextX = Math.max(8, Math.min(d.origX + dx, window.innerWidth - width - 8));
      const nextY = Math.max(8, Math.min(d.origY + dy, window.innerHeight - height - 8));
      (d.panel ? setPanelPos : setPos)({ x: nextX, y: nextY });
    };

    const handlePointerUp = () => {
      const wasMoved = bubbleDrag.current.moved;
      bubbleDrag.current.dragging = false;
      if (!wasMoved && !bubbleDrag.current.panel) {
        setExpanded(true);
      }
    };

    const handleTitleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
      if (floatingNode) {
        onUpdateNodeContent(floatingNode.id, e.target.value, floatingNode.content);
      }
    };

    const handleEditorInput = () => {
      if (editorRef.current && floatingNode) {
        onUpdateNodeContent(floatingNode.id, floatingNode.title, editorRef.current.innerHTML);
      }
    };

    const handlePopOut = async () => {
      if (!floatingNode) return;
      if (!('documentPictureInPicture' in window)) {
        onShowToast("Pop-out window requires desktop Chrome/Edge — using floating bubble pin!");
        return;
      }
      try {
        const pipWindow = await (window as any).documentPictureInPicture.requestWindow({
          width: Math.min(settings.expansionWidth, 480),
          height: Math.min(settings.expansionHeight, 520),
        });
        pipWindowRef.current?.close();
        pipWindowRef.current = pipWindow;

        const style = pipWindow.document.createElement('style');
        style.textContent = `
          body { margin:0; padding:18px;
> font-family:-apple-system,BlinkMacSystemFont,Inter,sans-serif; background: #171F45; color:
> #EDEFF7; }
          h2 { font-size:17px; font-weight:800; margin:0 0 12px 0; color: #FFC966; }
          .pip-body { font-size:14px; line-height:1.6; max-height:360px; overflow-y:auto; outline:
> none; }
          .pip-body img { max-width:100%; border-radius:8px; }
          .pip-btn { margin-top:14px; padding:8px 16px; border-radius:10px; border:none;
> background:#3A5FE0; color:#fff; font-weight:600; cursor:pointer; }
        `;
        pipWindow.document.head.appendChild(style);
```

src/components/BubblePin.tsx (continued from previous page)

```
198 |
199 |         const h = pipWindow.document.createElement('h2');
200 |         h.textContent = floatingNode.title || 'Untitled';
201 |
202 |         const body = pipWindow.document.createElement('div');
203 |         body.className = 'pip-body';
204 |         body.innerHTML = floatingNode.content || '';
205 |         body.contentEditable = 'true';
206 |         pipBodyRef.current = body;
207 |
208 |         body.oninput = () => {
209 |           const node = latestNode.current;
210 |           if (node) onUpdateNodeContent(node.id, node.title, body.innerHTML);
211 |         };
212 |
213 |         const btn = pipWindow.document.createElement('button');
214 |         btn.className = 'pip-btn';
215 |         btn.textContent = 'Copy content';
216 |         btn.onclick = () => {
217 |           const node = latestNode.current;
218 |           if (!node) return;
219 |           const text = `${node.title || 'Untitled'}\n\n${stripHtml(node.content)}`;
220 |           pipWindow.navigator.clipboard?.writeText(text);
221 |         };
222 |
223 |         pipWindow.document.body.appendChild(h);
224 |         pipWindow.document.body.appendChild(body);
225 |         pipWindow.document.body.appendChild(btn);
226 |
227 |         pipWindow.addEventListener('pagehide', () => {
228 |           pipWindowRef.current = null;
229 |           pipBodyRef.current = null;
230 |         });
231 |
232 |         setExpanded(false);
233 |       } catch (e) {
234 |         onShowToast("Couldn't open pop-out window");
235 |       }
236 |     };
237 |
238 |     return (
239 |       <>
240 |         {/* 1. Unexpanded Floating Bubble */}
241 |         {!expanded && (
242 |           <div
243 |             className="float-bubble-unique"
244 |             style={{
245 |               left: pos.x,
246 |               top: pos.y,
247 |               width: settings.bubblePopSize,
248 |               height: settings.bubblePopSize,
249 |               opacity: settings.transparency,
250 |             }}
251 |             onPointerDown={handlePointerDown}
252 |             onPointerMove={handlePointerMove}
253 |             onPointerUp={handlePointerUp}
254 |             onPointerCancel={() => { bubbleDrag.current.dragging = false; }}
255 |           >
256 |             <div className="bubble-glow-ring" />
257 |                       < s p a n   c l a s s N a m e = " f l o a t - b u b b l e - p i n - i c o n " >Ø=ÜÌ < /
258 |             <span className="float-bubble-title-text">
259 |               {(floatingNode.title || 'Untitled').slice(0, 10)}
260 |             </span>
261 |           </div>
262 |         )}
263 |
264 |         {/* 2. Expanded Floating Editable Panel */}
265 |         {expanded && (
266 |           <div
```

src/components/BubblePin.tsx (continued from previous page)

```
267 |             className="float-panel-editable"
268 |             ref={panelRef}
269 |             style={{
270 |               opacity: settings.transparency,
271 |               width: `min(90vw, ${settings.expansionWidth}px)`,
272 |               height: `min(85dvh, ${settings.expansionHeight}px)`,
273 |               maxWidth: `min(94vw, ${settings.expansionWidth}px)`,
274 |               maxHeight: `min(85dvh, ${settings.expansionHeight}px)`,
275 |               ...(panelPos ? { left: panelPos.x, top: panelPos.y, right: 'auto', bottom: 'auto' } :
    > |{}),
276 |             }}
277 |           >
278 |             {/* Header */}
279 |             <div className="float-panel-header"
280 |               onPointerDown={handlePointerDown} onPointerMove={handlePointerMove}
281 |               onPointerUp={() => { bubbleDrag.current.dragging = false; }}
282 |               onPointerCancel={() => { bubbleDrag.current.dragging = false; }}
283 |             >
284 |               <input
285 |                 className="float-panel-title-input"
286 |                 value={floatingNode.title}
287 |                 placeholder="Untitled note..."
288 |                 onChange={handleTitleChange}
289 |               />
290 |               <div className="float-panel-actions">
291 |                 <button
292 |                   className={`icon-btn mini ${settings.cropEnabled ? 'active-accent' : ''}`}
293 |                   onClick={() => onUpdateSettings({ cropEnabled: !settings.cropEnabled })}
294 |                   title="Toggle Viewport Crop"
295 |                 >
296 |                   <Crop size={15} />
297 |                 </button>
298 |                 <button
299 |                   className={`icon-btn mini ${showSettings ? 'active-accent' : ''}`}
300 |                   onClick={() => setShowSettings(!showSettings)}
301 |                   title="Bubble Settings"
302 |                 >
303 |                   <Settings size={15} />
304 |                 </button>
305 |                 <button
306 |                   className="icon-btn mini"
307 |                   onClick={() => onCopyNote(floatingNode)}
308 |                   title="Copy Note"
309 |                 >
310 |                   <Copy size={15} />
311 |                 </button>
312 |                 <button
313 |                   className="icon-btn mini"
314 |                   onClick={handlePopOut}
315 |                   title="Open desktop Picture-in-Picture window"
316 |                 >
317 |                   <ExternalLink size={15} />
318 |                 </button>
319 |                 <button
320 |                   className="icon-btn mini"
321 |                   onClick={() => {
322 |                     setExpanded(false);
323 |                     setShowSettings(false);
324 |                   }}
325 |                   title="Minimize to Bubble"
326 |                 >
327 |                   <Minimize2 size={15} />
328 |                 </button>
329 |                 <button className="icon-btn mini" onClick={onClose} title="Close Bubble">
330 |                   <X size={15} />
331 |                 </button>
332 |               </div>
333 |             </div>
334 |
```

src/components/BubblePin.tsx (continued from previous page)

```
335 |             {/* Settings Sub-Panel */}
336 |             {showSettings && (
337 |               <div className="bubble-settings-tray">
338 |                 <div className="tray-title">
339 |                   <span>Bubble Pin Customizer</span>
340 |                   <button
341 |                     className="reset-btn"
342 |                     onClick={() => onUpdateSettings(DEFAULT_BUBBLE_SETTINGS)}
343 |                   >
344 |                     <RotateCcw size={12} /> Reset
345 |                   </button>
346 |                 </div>
347 |
348 |                 <div className="setting-group">
349 |                   <label>Transparency ({Math.round(settings.transparency * 100)}%)</label>
350 |                   <input
351 |                     type="range"
352 |                     min="0.2"
353 |                     max="1.0"
354 |                     step="0.05"
355 |                     value={settings.transparency}
356 |                     onChange={(e) => onUpdateSettings({ transparency: parseFloat(e.target.value) })}
357 |                   />
358 |                 </div>
359 |
360 |                 <div className="setting-group">
361 |                   <label>Bubble Pop Size ({settings.bubblePopSize}px)</label>
362 |                   <div className="preset-pills">
363 |                     {[48, 64, 80, 96].map((sz) => (
364 |                       <button
365 |                         key={sz}
366 |                         className={`preset-pill ${settings.bubblePopSize === sz ? 'selected' : ''}`}
367 |                         onClick={() => onUpdateSettings({ bubblePopSize: sz })}
368 |                       >
369 |                         {sz === 48 ? 'S' : sz === 64 ? 'M' : sz === 80 ? 'L' : 'XL'}
370 |                       </button>
371 |                     ))}
372 |                   </div>
373 |                 </div>
374 |
375 |                 <div className="setting-group">
376 |                   <label>Expansion Size
    > |({settings.expansionWidth}x{settings.expansionHeight}px)</label>
377 |                   <div className="setting-dual">
378 |                     <div>
379 |                       <span>Width</span>
380 |                       <input
381 |                         type="range"
382 |                         min="300"
383 |                         max="720"
384 |                         step="20"
385 |                         value={settings.expansionWidth}
386 |                         onChange={(e) => onUpdateSettings({ expansionWidth: parseInt(e.target.value)
    > |})}
387 |                       />
388 |                     </div>
389 |                     <div>
390 |                       <span>Height</span>
391 |                       <input
392 |                         type="range"
393 |                         min="260"
394 |                         max="650"
395 |                         step="20"
396 |                         value={settings.expansionHeight}
397 |                         onChange={(e) => onUpdateSettings({ expansionHeight:
    > |parseInt(e.target.value) })}
398 |                       />
399 |                     </div>
400 |                   </div>
```

src/components/BubblePin.tsx (continued from previous page)

```
401 |                 </div>
402 |
403 |                 <div className="setting-group">
404 |                   <div className="flex-between">
405 |                     <label>Enable Crop Masking</label>
406 |                     <input
407 |                       type="checkbox"
408 |                       checked={settings.cropEnabled}
409 |                       onChange={(e) => onUpdateSettings({ cropEnabled: e.target.checked })}
410 |                     />
411 |                   </div>
412 |                   {settings.cropEnabled && (
413 |                     <div className="setting-dual">
414 |                       <div>
415 |                         <span>Top Offset ({settings.cropTop}px)</span>
416 |                         <input
417 |                           type="range"
418 |                           min="0"
419 |                           max="200"
420 |                           value={settings.cropTop}
421 |                           onChange={(e) => onUpdateSettings({ cropTop: parseInt(e.target.value) })}
422 |                         />
423 |                       </div>
424 |                       <div>
425 |                         <span>Visible Height ({settings.cropHeight}px)</span>
426 |                         <input
427 |                           type="range"
428 |                           min="100"
429 |                           max="400"
430 |                           value={settings.cropHeight}
431 |                           onChange={(e) => onUpdateSettings({ cropHeight: parseInt(e.target.value)
    > |})}
432 |                         />
433 |                       </div>
434 |                     </div>
435 |                   )}
436 |                 </div>
437 |               </div>
438 |             )}
439 |
440 |             {/* Editable Content Area with Optional Crop Masking */}
441 |             <div
442 |               className={`float-panel-scroll-container ${settings.cropEnabled ? 'cropped-mask' :
    > |''}`}
443 |               style={
444 |                 settings.cropEnabled
445 |                   ? {
446 |                       height: `${settings.cropHeight}px`,
447 |                       maxHeight: `${settings.cropHeight}px`,
448 |                       flex: '0 1 auto',
449 |                     }
450 |                   : {}
451 |               }
452 |             >
453 |               <div
454 |                 ref={editorRef}
455 |                 className="float-panel-body-editable"
456 |                 contentEditable
457 |                 suppressContentEditableWarning
458 |                 onInput={handleEditorInput}
459 |                 placeholder="Start writing inside floating bubble..."
460 |               />
461 |             </div>
462 |           </div>
463 |         )}
464 |       </>
465 |     );
466 |   };
467 |
```

src/components/ActionSheet.tsx

UI Components | 76 Lines

Role: Context action sheet for pin, unpin, copy, PDF, and trash

```tsx
import React from 'react';
import { Pin, Copy, Download, Archive, Trash2, FileCode } from 'lucide-react';
import { NoteNode } from '../types';

interface ActionSheetProps {
  longPressNode: NoteNode | null;
  onClose: () => void;
  onTogglePin: (id: string) => void;
  onCopyNote: (node: NoteNode) => void;
  onDownloadPdf: (node: NoteNode) => void;
  onArchive: (id: string) => void;
  onDelete: (id: string) => void;
  onOpenCodeExport?: () => void;
}

export const ActionSheet: React.FC<ActionSheetProps> = ({
  longPressNode,
  onClose,
  onTogglePin,
  onCopyNote,
  onDownloadPdf,
  onArchive,
  onDelete,
  onOpenCodeExport,
}) => {
  if (!longPressNode) return null;

  return (
    <div className="sheet-backdrop" onClick={onClose}>
      <div className="sheet" onClick={(e) => e.stopPropagation()}>
        <div className="sheet-title">{longPressNode.title || 'Untitled Note'}</div>

        <button
          className={`sheet-item ${longPressNode.pinned ? 'active-pin' : ''}`}
          onClick={() => onTogglePin(longPressNode.id)}
        >
          <Pin size={16} />
          {longPressNode.pinned ? 'Unpin note (Return to normal position)' : 'Pin note (Move to top by time
> | pinned)'}
        </button>

        <button className="sheet-item" onClick={() => onCopyNote(longPressNode)}>
          <Copy size={16} /> Copy content
        </button>

        <button className="sheet-item" onClick={() => onDownloadPdf(longPressNode)}>
          <Download size={16} /> Download Note as PDF
        </button>

        {onOpenCodeExport && (
          <button
            className="sheet-item"
            onClick={() => {
              onClose();
              onOpenCodeExport();
            }}
          >
            <FileCode size={16} /> Download Project Code as PDF
          </button>
        )}

        <button className="sheet-item" onClick={() => onArchive(longPressNode.id)}>
          <Archive size={16} /> Archive note
        </button>

```

src/components/ActionSheet.tsx (continued from previous page)

```
          <button className="sheet-item danger" onClick={() => onDelete(longPressNode.id)}>
            <Trash2 size={16} /> Delete to Recycle Bin
          </button>

          <button className="sheet-cancel" onClick={onClose}>
            Cancel
          </button>
        </div>
      </div>
    );
  };

```

src/components/SearchModal.tsx

UI Components | 114 Lines

Role: Global full-text search engine with highlighted matched snippets

```tsx
import React, { useState, useMemo, useRef, useEffect } from 'react';
import { Search, X } from 'lucide-react';
import { NoteNode, SearchResult } from '../types';
import { collectSearchResults, pathTitles } from '../utils/tree';

interface SearchModalProps {
  tree: NoteNode;
  isOpen: boolean;
  onClose: () => void;
  onSelectResult: (result: SearchResult, query: string) => void;
}

export const SearchModal: React.FC<SearchModalProps> = ({
  tree,
  isOpen,
  onClose,
  onSelectResult,
}) => {
  const [searchQuery, setSearchQuery] = useState('');
  const inputRef = useRef<HTMLInputElement>(null);

  useEffect(() => {
    if (isOpen) {
      const timer = setTimeout(() => inputRef.current?.focus(), 50);
      return () => clearTimeout(timer);
    } else {
      setSearchQuery('');
    }
  }, [isOpen]);

  const searchResults = useMemo(() => {
    const q = searchQuery.trim();
    if (!q || !tree) return [];
    const out: SearchResult[] = [];
    collectSearchResults(tree, q, [], out);
    return out.slice(0, 60);
  }, [searchQuery, tree]);

  if (!isOpen) return null;

  return (
    <div className="search-backdrop" onClick={onClose}>
      <div className="search-panel" onClick={(e) => e.stopPropagation()}>
        <div className="search-input-row">
          <Search size={18} />
          <input
            ref={inputRef}
            className="search-input"
            placeholder="Search all notes..."
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
            onKeyDown={(e) => {
              if (e.key === 'Escape') onClose();
              if (e.key === 'Enter' && searchResults.length > 0) {
                onSelectResult(searchResults[0], searchQuery.trim());
              }
            }}
          />
          <button className="search-close-btn" onClick={onClose}>
            <X size={16} />
          </button>
        </div>

        <div className="search-results">
          {searchQuery.trim() === '' ? (
```

src/components/SearchModal.tsx (continued from previous page)

```
              <div className="search-empty">Type any word to find notes instantly</div>
            ) : searchResults.length === 0 ? (
              <div className="search-empty">No notes match “{searchQuery.trim()}”</div>
            ) : (
              searchResults.map((r) => {
                const crumbs = pathTitles(tree, r.path);
                const crumbText = crumbs.length ? `Home / ${crumbs.join(' / ')}` : 'Home';
                const q = searchQuery.trim();

                let titleNode: React.ReactNode = r.title;
                const qIdx = r.title.toLowerCase().indexOf(q.toLowerCase());
                if (r.matchType === 'title' && qIdx !== -1) {
                  titleNode = (
                    <>
                      {r.title.slice(0, qIdx)}
                      <mark className="result-hit">{r.title.slice(qIdx, qIdx + q.length)}</mark>
                      {r.title.slice(qIdx + q.length)}
                    </>
                  );
                }

                return (
                  <button
                    key={r.id}
                    className="search-result-item"
                    onClick={() => onSelectResult(r, q)}
                  >
                    <div className="search-result-head">
                      <div className="search-result-title">{titleNode}</div>
                      {r.pinned && <span className="search-result-tag">PINNED</span>}
                      {r.archived && <span className="search-result-tag">ARCHIVED</span>}
                    </div>
                    {r.snippet && <div className="search-result-snippet">{
                      (() => {
                        const index = r.snippet.toLowerCase().indexOf(q.toLowerCase());
                        return index < 0 ? r.snippet : <>{r.snippet.slice(0, index)}<mark
> |className="result-hit">{r.snippet.slice(index, index + q.length)}</mark>{r.snippet.slice(index +
> |q.length)}</>;
                      })()
                    }</div>}
                    <div className="search-result-path">{crumbText}</div>
                  </button>
                );
              })
            )}
          </div>
        </div>
      </div>
    );
  };

```

src/components/ArchiveTrashView.tsx

UI Components | 119 Lines

Role: Recycle bin and archive manager with parent restore flow

```tsx
import React from 'react';
import { ArchiveRestore, RotateCcw, Trash2 } from 'lucide-react';
import { NoteNode, TrashEntry } from '../types';
import { findNode } from '../utils/tree';

interface ArchiveTrashViewProps {
  mode: 'archive' | 'trash';
  currentNode: NoteNode;
  tree: NoteNode;
  trash: TrashEntry[];
  trashAreaId: string;
  onRestoreArchived: (id: string) => void;
  onRestoreTrash: (trashId: string) => void;
  onPermanentDelete: (trashId: string) => void;
  onDeleteNote: (id: string) => void;
}

export const ArchiveTrashView: React.FC<ArchiveTrashViewProps> = ({
  mode,
  currentNode,
  tree,
  trash,
  trashAreaId,
  onRestoreArchived,
  onRestoreTrash,
  onPermanentDelete,
  onDeleteNote,
}) => {
  if (mode === 'archive') {
    const archivedChildren = (currentNode.children || []).filter((c) => c.archived);

    return (
      <div className="board-scroll">
        <div className="board-inner">
          {archivedChildren.length === 0 ? (
            <div className="board-empty">
              <p>No archived notes in this page</p>
            </div>
          ) : (
            archivedChildren.map((child) => {
              const innerCount = (child.children || []).filter((c) => !c.archived).length;
              return (
                <div className="archive-row" key={child.id}>
                  <div className="archive-row-title">
                    {child.title || 'Untitled Note'}
                    <span>
                      {innerCount} nested note{innerCount === 1 ? '' : 's'} inside
                    </span>
                  </div>
                  <button
                    className="archive-mini-btn"
                    onClick={() => onRestoreArchived(child.id)}
                    title="Restore to active page"
                  >
                    <ArchiveRestore size={16} />
                  </button>
                  <button
                    className="archive-mini-btn danger"
                    onClick={() => onDeleteNote(child.id)}
                    title="Move to Recycle Bin"
                  >
                    <Trash2 size={16} />
                  </button>
                </div>
              );
```

src/components/ArchiveTrashView.tsx (continued from previous page)

```
66 |               })
67 |             )}
68 |           </div>
69 |         </div>
70 |       );
71 |     }
72 |
73 |     // Trash Mode
74 |     const areaTrash = trash.filter((t) => t.parentId === trashAreaId || (trashAreaId === 'root' &&
   > |!findNode(tree, t.parentId)));
75 |
76 |     return (
77 |       <div className="board-scroll">
78 |         <div className="board-inner">
79 |           {areaTrash.length === 0 ? (
80 |             <div className="board-empty">
81 |               <p>Recycle bin is empty for this note area</p>
82 |             </div>
83 |           ) : (
84 |             areaTrash.map((entry) => (
85 |               <div className="archive-row" key={entry.id}>
86 |                 <div className="archive-row-title">
87 |                   {entry.node.title || 'Untitled Note'}
88 |                   {entry.node.pinned && <span className="pinned-tag">PINNED NOTE</span>}
89 |                   <span>
90 |                     Deleted {new Date(entry.deletedAt).toLocaleDateString()}
91 |                     {entry.node.children && entry.node.children.length > 0
92 |                       ? ` · ${entry.node.children.length} nested note${
93 |                           entry.node.children.length === 1 ? '' : 's'
94 |                         } inside will restore too`
95 |                       : ''}
96 |                   </span>
97 |                 </div>
98 |                 <button
99 |                   className="archive-mini-btn"
100 |                   onClick={() => onRestoreTrash(entry.id)}
101 |                   title="Restore to original parent page"
102 |                 >
103 |                   <RotateCcw size={16} />
104 |                 </button>
105 |                 <button
106 |                   className="archive-mini-btn danger"
107 |                   onClick={() => onPermanentDelete(entry.id)}
108 |                   title="Delete forever"
109 |                 >
110 |                   <Trash2 size={16} />
111 |                 </button>
112 |               </div>
113 |             ))
114 |           )}
115 |         </div>
116 |       </div>
117 |     );
118 |   };
119 |
```

src/components/CodeExportModal.tsx

UI Components | 348 Lines

Role: In-app codebase browser, file viewer, and PDF downloader

```tsx
import React, { useState } from 'react';
import {
  FileCode,
  Download,
  Copy,
  Check,
  X,
  FileText,
  ExternalLink,
  Sparkles,
  BookOpen,
  Code,
  Archive,
} from 'lucide-react';
import {
  CODEBASE_FILES,
  CORE_FEATURES_LIST,
  SourceFile,
  generateClientCodePdf,
} from '../utils/codeExport';

interface CodeExportModalProps {
  isOpen: boolean;
  onClose: () => void;
  onShowToast: (msg: string) => void;
}

export const CodeExportModal: React.FC<CodeExportModalProps> = ({
  isOpen,
  onClose,
  onShowToast,
}) => {
  const [activeTab, setActiveTab] = useState<'files' | 'features'>('files');
  const [selectedFile, setSelectedFile] = useState<SourceFile>(CODEBASE_FILES[0]);
  const [copiedPath, setCopiedPath] = useState<string | null>(null);
  const [isGenerating, setIsGenerating] = useState(false);

  if (!isOpen) return null;

  const totalLines = CODEBASE_FILES.reduce((acc, f) => acc + f.lines, 0);

  const handleDownloadPdf = async () => {
    setIsGenerating(true);
    onShowToast('Downloading comprehensive Codebase PDF...');
    try {
      const link = document.createElement('a');
      link.href = `${import.meta.env.BASE_URL}Pinboard_Notes_Codebase.pdf`;
      link.download = 'Pinboard_Notes_Codebase.pdf';
      document.body.appendChild(link);
      link.click();
      document.body.removeChild(link);
      onShowToast('Download started: Pinboard_Notes_Codebase.pdf');
    } catch (err) {
      try {
        generateClientCodePdf();
        onShowToast('PDF generated and downloaded successfully!');
      } catch (clientErr) {
        onShowToast('Could not download PDF');
      }
    } finally {
      setTimeout(() => setIsGenerating(false), 900);
    }
  };

  const handleDownloadZip = () => {
```

src/components/CodeExportModal.tsx (continued from previous page)

```
66 |       onShowToast('Downloading Pinboard_Notes_Complete.zip...');
67 |       try {
68 |         const link = document.createElement('a');
69 |         link.href = `${import.meta.env.BASE_URL}Pinboard_Notes_Complete.zip`;
70 |         link.download = 'Pinboard_Notes_Complete.zip';
71 |         document.body.appendChild(link);
72 |         link.click();
73 |         document.body.removeChild(link);
74 |         onShowToast('Download started: Pinboard_Notes_Complete.zip');
75 |       } catch (err) {
76 |         onShowToast('Could not download zip');
77 |       }
78 |     };
79 |
80 |     const handleCopyCode = async (file: SourceFile) => {
81 |       try {
82 |         await navigator.clipboard.writeText(file.content);
83 |         setCopiedPath(file.path);
84 |         onShowToast(`Copied ${file.path} to clipboard`);
85 |         setTimeout(() => setCopiedPath(null), 2000);
86 |       } catch (e) {
87 |         onShowToast('Failed to copy to clipboard');
88 |       }
89 |     };
90 |
91 |     const handleDownloadSingleFile = (file: SourceFile) => {
92 |       const blob = new Blob([file.content], { type: 'text/plain;charset=utf-8' });
93 |       const url = URL.createObjectURL(blob);
94 |       const a = document.createElement('a');
95 |       a.href = url;
96 |       a.download = file.path.split('/').pop() || 'code.txt';
97 |       document.body.appendChild(a);
98 |       a.click();
99 |       document.body.removeChild(a);
100 |       URL.revokeObjectURL(url);
101 |       onShowToast(`Downloaded ${file.path}`);
102 |     };
103 |
104 |     return (
105 |       <div className="sheet-backdrop" onClick={onClose} style={{ zIndex: 1100 }}>
106 |         <div
107 |           className="code-export-modal"
108 |           onClick={(e) => e.stopPropagation()}
109 |           role="dialog"
110 |           aria-modal="true"
111 |         >
112 |           {/* Modal Header */}
113 |           <div className="code-export-header">
114 |             <div className="code-export-header-title">
115 |               <div className="code-badge-icon">
116 |                 <FileCode size={20} />
117 |               </div>
118 |               <div>
119 |                 <h3>Complete Codebase & Architectural Specification</h3>
120 |                 <p>
121 |                   10 Core Feature Modules • {CODEBASE_FILES.length} Files •
    > | {totalLines.toLocaleString()} Lines of Code
122 |                 </p>
123 |               </div>
124 |             </div>
125 |
126 |             <div className="code-export-header-actions">
127 |               <button
128 |                 className="export-zip-btn"
129 |                 onClick={handleDownloadZip}
130 |                 title="Download entire project repository as a ZIP archive (includes PDF + all
    > | source files)"
131 |               >
132 |                 <Archive size={16} />
```

src/components/CodeExportModal.tsx (continued from previous page)

```
133 |                 <span>Download ZIP</span>
134 |               </button>
135 |
136 |               <button
137 |                 className="export-pdf-main-btn"
138 |                 onClick={handleDownloadPdf}
139 |                 disabled={isGenerating}
140 |                 title="Download the full architectural specification & unabridged codebase PDF"
141 |               >
142 |                 <Download size={16} />
143 |                 <span>{isGenerating ? 'Preparing PDF...' : 'Download PDF'}</span>
144 |               </button>
145 |
146 |               <button className="icon-btn" onClick={onClose} title="Close modal">
147 |                 <X size={18} />
148 |               </button>
149 |             </div>
150 |           </div>
151 |
152 |           {/* Navigation Tabs */}
153 |           <div style={{ display: 'flex', borderBottom: '1px solid var(--divider)', padding: '0
    > |16px', background: 'rgba(0,0,0,0.03)' }}>
154 |             <button
155 |               onClick={() => setActiveTab('files')}
156 |               style={{
157 |                 display: 'flex',
158 |                 alignItems: 'center',
159 |                 gap: 8,
160 |                 padding: '12px 18px',
161 |                 border: 'none',
162 |                 background: 'transparent',
163 |                 color: activeTab === 'files' ? 'var(--accent1)' : 'var(--text-dim)',
164 |                 borderBottom: activeTab === 'files' ? '2px solid var(--accent1)' : '2px solid
    > |transparent',
165 |                 fontWeight: 600,
166 |                 fontSize: 13,
167 |                 cursor: 'pointer',
168 |               }}
169 |             >
170 |               <Code size={15} />
171 |               <span>Source Code Files ({CODEBASE_FILES.length})</span>
172 |             </button>
173 |             <button
174 |               onClick={() => setActiveTab('features')}
175 |               style={{
176 |                 display: 'flex',
177 |                 alignItems: 'center',
178 |                 gap: 8,
179 |                 padding: '12px 18px',
180 |                 border: 'none',
181 |                 background: 'transparent',
182 |                 color: activeTab === 'features' ? 'var(--accent1)' : 'var(--text-dim)',
183 |                 borderBottom: activeTab === 'features' ? '2px solid var(--accent1)' : '2px solid
    > |transparent',
184 |                 fontWeight: 600,
185 |                 fontSize: 13,
186 |                 cursor: 'pointer',
187 |               }}
188 |             >
189 |               <BookOpen size={15} />
190 |               <span>Feature Specifications ({CORE_FEATURES_LIST.length})</span>
191 |             </button>
192 |           </div>
193 |
194 |           {/* Tab 1: Source Files */}
195 |           {activeTab === 'files' && (
196 |             <div className="code-export-body">
197 |               {/* File sidebar list */}
198 |               <div className="code-file-list">
```

src/components/CodeExportModal.tsx (continued from previous page)

```
                <div className="code-list-title">Repository Files</div>
                {CODEBASE_FILES.map((file) => {
                  const isSelected = selectedFile.path === file.path;
                  return (
                    <button
                      key={file.path}
                      className={`code-file-item ${isSelected ? 'active' : ''}`}
                      onClick={() => setSelectedFile(file)}
                    >
                      <div className="file-item-left">
                        <FileText size={15} className="file-icon" />
                        <div className="file-info">
                          <span className="file-name">{file.path.split('/').pop()}</span>
                          <span className="file-path">{file.path}</span>
                        </div>
                      </div>
                      <div className="file-meta">
                        <span className="file-badge">{file.category}</span>
                        <span className="file-lines">{file.lines}L</span>
                      </div>
                    </button>
                  );
                })}
              </div>

              {/* Code viewer */}
              <div className="code-preview-pane">
                <div className="code-pane-toolbar">
                  <div className="pane-file-details">
                    <span className="pane-filename">{selectedFile.path}</span>
                    <span className="pane-desc">{selectedFile.description}</span>
                  </div>
                  <div className="pane-actions">
                    <button
                      className="pane-btn"
                      onClick={() => handleCopyCode(selectedFile)}
                      title="Copy file source code"
                    >
                      {copiedPath === selectedFile.path ? <Check size={14} /> : <Copy size={14} />}
                      <span>{copiedPath === selectedFile.path ? 'Copied' : 'Copy'}</span>
                    </button>
                    <button
                      className="pane-btn"
                      onClick={() => handleDownloadSingleFile(selectedFile)}
                      title="Download this file"
                    >
                      <Download size={14} />
                      <span>Download File</span>
                    </button>
                  </div>
                </div>

                <pre className="code-viewer-content">
                  <code>
                    {selectedFile.content.split('\n').map((line, idx) => (
                      <div key={idx} className="code-line">
                        <span className="code-line-number">{idx + 1}</span>
                        <span className="code-line-text">{line || ' '}</span>
                      </div>
                    ))}
                  </code>
                </pre>
              </div>
            </div>
          )}

          {/* Tab 2: Feature Specifications */}
          {activeTab === 'features' && (
            <div style={{ flex: 1, overflowY: 'auto', padding: '24px 32px' }}>
```

src/components/CodeExportModal.tsx (continued from previous page)

```
              <div style={{ maxWidth: 860, margin: '0 auto' }}>
                <div style={{ marginBottom: 20 }}>
                  <h4 style={{ margin: 0, fontSize: 16, fontWeight: 700, color: 'var(--text)' }}>
                    Architectural & Functional Blueprint (10 Modules)
                  </h4>
                  <p style={{ margin: '4px 0 0', fontSize: 13, color: 'var(--text-dim)' }}>
                    Every feature engineered to specification, including tree mutations,
> drag-reorder physics, floating bubble parameters, and Recycle Bin landing flows.
                  </p>
                </div>

                <div style={{ display: 'grid', gridTemplateColumns: '1fr', gap: 16 }}>
                  {CORE_FEATURES_LIST.map((feat) => (
                    <div
                      key={feat.id}
                      style={{
                        background: 'var(--card-bg)',
                        border: '1px solid var(--card-border)',
                        borderRadius: 14,
                        padding: '16px 20px',
                      }}
                    >
                      <div style={{ display: 'flex', alignItems: 'center', gap: 10, marginBottom: 8
> }}>
                        <span
                          style={{
                            background: 'var(--pill-bg)',
                            border: '1px solid var(--pill-border)',
                            color: 'var(--accent1)',
                            fontSize: 11,
                            fontWeight: 800,
                            padding: '2px 8px',
                            borderRadius: 6,
                          }}
                        >
                          {feat.id}
                        </span>
                        <span style={{ fontSize: 14, fontWeight: 700, color: 'var(--text)' }}>
                          {feat.title}
                        </span>
                      </div>
                      <p style={{ margin: 0, fontSize: 12.5, lineHeight: 1.6, color:
> 'var(--text-dim)' }}>
                        {feat.desc}
                      </p>
                    </div>
                  ))}
                </div>
              </div>
            </div>
          )}

          {/* Modal Footer */}
          <div className="code-export-footer">
            <div className="footer-left">
              <span className="pdf-info-pill">
                <Sparkles size={13} />
                Includes complete PDF document, 10 feature blueprints, and full source code
              </span>
            </div>
            <div className="footer-right" style={{ display: 'flex', gap: 10 }}>
              <a
                href={`${import.meta.env.BASE_URL}Pinboard_Notes_Complete.zip`}
                download="Pinboard_Notes_Complete.zip"
                className="direct-pdf-link"
                title="Download entire codebase and PDF bundled as a ZIP file"
              >
                Direct ZIP Link <Archive size={13} />
              </a>
```

src/components/CodeExportModal.tsx (continued from previous page)

```
334 |               <a
335 |                 href={`${import.meta.env.BASE_URL}Pinboard_Notes_Codebase.pdf`}
336 |                 download="Pinboard_Notes_Codebase.pdf"
337 |                 className="direct-pdf-link"
338 |                 title="Direct link to static codebase PDF file"
339 |               >
340 |                 Direct PDF <ExternalLink size={13} />
341 |               </a>
342 |             </div>
343 |           </div>
344 |         </div>
345 |       </div>
346 |     );
347 |   };
348 |
```

scripts/generate_code_pdf.ts

Build Scripts | 539 Lines

Role: Comprehensive PDF compiler script for project code & documentation

```
import fs from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';
import { jsPDF } from 'jspdf';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const PROJECT_ROOT = path.resolve(__dirname, '..');
const OUTPUT_PATH = path.resolve(PROJECT_ROOT, 'public', 'Pinboard_Notes_Codebase.pdf');

interface FileSpec {
  path: string;
  category: string;
  description: string;
}

const FILES_TO_INCLUDE: FileSpec[] = [
  { path: 'metadata.json', category: 'Configuration', description: 'Application metadata, identity, and studio capabilities' },
  { path: 'package.json', category: 'Configuration', description: 'NPM package dependencies, scripts, and build tasks' },
  { path: 'vite.config.ts', category: 'Build Setup', description: 'Vite build tool configuration and Tailwind CSS integration' },
  { path: 'tsconfig.json', category: 'Configuration', description: 'TypeScript compiler configuration and strict path aliases' },
  { path: 'index.html', category: 'Web Entry', description: 'HTML entry point with viewport, title, and OpenGraph tags' },
  { path: 'src/main.tsx', category: 'Web Entry', description: 'React 19 entry point and root DOM container mounting' },
  { path: 'src/types.ts', category: 'Data Models', description: 'Type definitions for NoteNode, TrashEntry, and BubbleSettings' },
  { path: 'src/vite-env.d.ts', category: 'Types', description: 'Vite environment and module declaration for raw imports' },
  { path: 'src/App.tsx', category: 'Application Core', description: 'Main application controller, state machine, and PDF integration' },
  { path: 'src/index.css', category: 'Design System', description: 'Complete CSS variables, dark/light themes, and animation rules' },
  { path: 'src/utils/storage.ts', category: 'Persistence', description: 'Local storage engine for durable cross-session data storage' },
  { path: 'src/utils/tree.ts', category: 'Tree Algorithms', description: 'Tree hierarchy manipulation, sorting, pathing, and chip sync' },
  { path: 'src/utils/codeExport.ts', category: 'Code Export', description: 'In-browser codebase catalog and client-side PDF compiler' },
  { path: 'src/components/TopBar.tsx', category: 'UI Components', description: 'Navigation top bar, breadcrumb stack, and global tools' },
  { path: 'src/components/BoardView.tsx', category: 'UI Components', description: 'Interactive board separating pinned hierarchy and regular notes' },
  { path: 'src/components/NoteCard.tsx', category: 'UI Components', description: 'Interactive card with hold-to-reorder and long-press menu' },
  { path: 'src/components/EditorView.tsx', category: 'UI Components', description: 'Rich text writing canvas, formatting toolbar, and nested chips' },
  { path: 'src/components/BubblePin.tsx', category: 'UI Components', description: 'Detached editable floating bubble pin with viewport cropping' },
  { path: 'src/components/ActionSheet.tsx', category: 'UI Components', description: 'Context action sheet for pin, unpin, copy, PDF, and trash' },
  { path: 'src/components/SearchModal.tsx', category: 'UI Components', description: 'Global full-text search engine with highlighted matched snippets' },
  { path: 'src/components/ArchiveTrashView.tsx', category: 'UI Components', description: 'Recycle bin and archive manager with parent restore flow' },
  { path: 'src/components/CodeExportModal.tsx', category: 'UI Components', description: 'In-app codebase browser, file viewer, and PDF downloader' },
  { path: 'scripts/generate_code_pdf.ts', category: 'Build Scripts', description: 'Comprehensive PDF compiler script for project code & documentation' },
  ];

```

scripts/generate_code_pdf.ts (continued from previous page)

```
43 |  interface FeatureSpec {
44 |    id: string;
45 |    title: string;
46 |    subtitle: string;
47 |    details: string[];
48 |  }
49 |
50 |  const FEATURE_SPECS: FeatureSpec[] = [
51 |    {
52 |      id: 'F01',
53 |      title: 'Hierarchical Tree Architecture & Path Navigation',
54 |      subtitle: 'Multi-level nested note trees with immutable updates and breadcrumb navigation',
55 |      details: [
56 |        'Data Model: Every note is represented as a recursive NoteNode containing id, title, content
   > |(HTML string), archived (boolean), pinned (boolean), pinnedAt (timestamp | null), order (number),
   > |and children (NoteNode[]).',
57 |        'Navigation Stack: Navigation state is maintained via a string array stack: string[]
   > |representing the active path from "root" to the current node. Breadcrumbs display the ancestral
   > |trail.',
58 |        'Immutable Mutations: All tree updates use deep cloning (cloneTree) ensuring pure React
   > |state transitions, zero side-effects, and reliable re-rendering.',
59 |        'Auto-Persistence: State modifications are immediately synced to localStorage; optional
   > |host-storage writes are serialized to protect recent edits.',
60 |      ],
61 |    },
62 |    {
63 |      id: 'F02',
64 |      title: 'Hierarchy of Importance: Pinning & Chronological Sorting',
65 |      subtitle: 'Pinned notes elevate with structural priority and chronological ordering',
66 |      details: [
67 |        'Significance Over Glow: Pinned notes are not merely highlighted with a visual glow; they
   > |are given explicit organizational hierarchy at the top of the container.',
68 |        'Chronological Timestamping: When a note is pinned, node.pinnedAt is set to Date.now().
   > |Pinned notes are dynamically sorted descending by pinnedAt, so the most recently pinned note
   > |always appears first.',
69 |        'Clear Visual Separation: BoardView splits items into two distinct sections: "Pinned Notes
   > |(Hierarchy of Importance)" at the top with a distinct pin badge, and "Notes" below.',
70 |        'Graceful Unpinning: When unpinned, node.pinned is set to false and node.pinnedAt is reset
   > |to null. The note returns back to its standard order in the regular notes section without data
   > |loss.',
71 |      ],
72 |    },
73 |    {
74 |      id: 'F03',
75 |      title: 'Hold & Arrange Notes\' Order (Manual Drag & Drop)',
76 |      subtitle: 'Touch & pointer-driven hold-and-drag reordering for custom prioritization',
77 |      details: [
78 |        'Grip Handles & Pointer Events: Each NoteCard features a dedicated drag grip handle
   > |(GripVertical) and handles onPointerDown/onDragStart events.',
79 |        'Drop Indicator & Animation: While dragging, cards calculate drag index and render visual
   > |insertion guides with drop target styling.',
80 |        'Persistent Reorder Algorithm: The reorderChildren utility mutates the order property of
   > |child nodes and updates the parent tree in state, ensuring customized ordering persists across
   > |reloads.',
81 |        'Scoped Dragging: Reordering works independently within both the pinned section and the
   > |unpinned section, giving users full layout autonomy.',
82 |      ],
83 |    },
84 |    {
85 |      id: 'F04',
86 |      title: 'Editable Floating Bubble Pin & Customizable Viewport',
87 |      subtitle: 'Detached, floating, resizable and croppable note workspace',
88 |      details: [
89 |        'Detached Multitasking: Any note can be popped out into a floating bubble pin that hovers
   > |above the entire application.',
90 |        'Full Bi-directional Editing: The bubble pin is completely editable with a rich text area.
   > |Typing inside the bubble immediately mutates the underlying tree node via onUpdateNodeContent.',
91 |        'Comprehensive Settings Panel: A toggleable settings modal on the bubble pin provides
   > |dynamic sliders for:',
```

```
92 |         '  • Transparency Slider: Adjusts container opacity from 20% (see-through reference) to 100%
   > |(solid focus).',
93 |         '  • Bubble Pop Size: Adjusts the collapsed/mini badge trigger diameter from 48px to 96px.',
94 |         '  • Expansion Size: Controls expanded panel width (320px to 720px) and height (280px to
   > |620px).',
95 |         '  • Viewport Cropping Engine: When enabled, users can crop top and bottom insets to
   > |restrict the visible window exclusively to a specific line, code snippet, or paragraph.',
96 |         'Draggable Physics: Users can drag the bubble pin anywhere across the viewport via pointer
   > |drag on the header handle.',
97 |       ],
98 |     },
99 |     {
100 |       id: 'F05',
101 |       title: 'Unique Standout Bubble Pop Button',
102 |       subtitle: 'Distinctive visual design and identity for the bubble pop feature',
103 |       details: [
104 |         'Distinct Identity: The Bubble Pop trigger is visually differentiated from the standard Pin+
   > |button.',
105 |         'Vibrant Gradient & Halo: Styled with a radiant linear gradient (linear-gradient(135deg,
   > |#FF6B6B, #9B51E0)), subtle ambient drop-shadow, and pulsing halo.',
106 |         'Animated Sparkle Icon: Houses an animated Sparkles icon with gentle rotation and a clear
   > |"Bubble Pin" text label in writing mode.',
107 |         'Instant Elevation: One click transforms any active note into an editable detached floating
   > |bubble pin.',
108 |       ],
109 |     },
110 |     {
111 |       id: 'F06',
112 |       title: 'Recycle Bin & Pinned Note Restoration Flow',
113 |       subtitle: 'Safe deletion, parent tracking, and smart contextual restoration',
114 |       details: [
115 |         'Parent Lineage Preservation: When a note is deleted, it is placed in the trash array as a
   > |TrashEntry with its parentId, deletedAt timestamp, and deep-cloned node data.',
116 |         'Re-embedding Content Chips: Upon restoration, the note is re-inserted into parent.children.
   > |If parent.content lacks the interactive chip, makeChipHtml re-embeds the note chip into the parent
   > |note content.',
117 |         'Parent Page Landing Logic: Restoring a pinned note correctly routes the user when clicking
   > |the parent page to the parent pinned note\'s writing page (where the restored note is pinned as a
   > |chip), rather than entering directly into the restored note.',
118 |         'Permanent Deletion: Items in the Recycle Bin can be selectively restored or permanently
   > |expunged.',
119 |       ],
120 |     },
121 |     {
122 |       id: 'F07',
123 |       title: 'Archive System with Instant Restoration',
124 |       subtitle: 'Declutter boards by archiving completed or reference notes',
125 |       details: [
126 |         'Soft Archival: Setting node.archived = true hides notes from the active board without
   > |altering their internal hierarchy.',
127 |         'Dedicated Archive View: Users can view all archived notes for the current page container,
   > |complete with date tags and pin badges.',
128 |         'One-Click Restoration: Unarchiving seamlessly restores the note to the active board and
   > |ensures chips in parent content are synced.',
129 |       ],
130 |     },
131 |     {
132 |       id: 'F08',
133 |       title: 'Instant Global Full-Text Search',
134 |       subtitle: 'Comprehensive search across titles and rich text with highlighted matches',
135 |       details: [
136 |         'Full Tree Traversal: The search engine recursively scans every node in the tree
   > |hierarchy.',
137 |         'HTML Tag Stripping: Content is stripped of HTML tags before querying, ensuring clean,
   > |accurate text comparisons.',
138 |         'Snippet Context Generation: Matches generate contextual text excerpts with the exact search
   > |query emphasized in highlighted badges.',
139 |         'Deep Stack Reconstruction: Clicking any search result reconstructs the complete parent path
   > |array (stack: result.path) and transitions immediately to writing mode.',
```

scripts/generate_code_pdf.ts (continued from previous page)

```
      ],
    },
    {
      id: 'F09',
      title: 'Dual PDF Export Engines (Note & Full Codebase)',
      subtitle: 'High-fidelity document generation powered by jsPDF',
      details: [
        'Note-Level PDF Export: Users can export any note into a clean, formatted A4 PDF containing
> |its title, metadata, and stripped body text.',
        'Complete Codebase PDF Generator: Bundles every source file, configuration file, build
> |script, and architectural documentation into a single comprehensive PDF.',
        'Table of Contents & Line Numbering: Code files are rendered with monospace typography, line
> |numbering, file metadata badges, and running headers/footers.',
        'Static & Dynamic Availability: Pre-generated to public/Pinboard_Notes_Codebase.pdf and
> |dynamically downloadable through the in-app modal.',
      ],
    },
    {
      id: 'F10',
      title: 'Design System & Responsive Dual Themes',
      subtitle: 'Midnight Dark and Warm Parchment Light modes with CSS variable tokens',
      details: [
        'Midnight Dark Theme: High contrast deep blue palette (--bg1: #0A1024, --bg2: #131B3B,
> |--accent1: #3A5FE0, --pin-active: #FFC966).',
        'Warm Parchment Light Theme: Paper-inspired aesthetic (--bg1: #F3EAD9, --bg2: #F6EFE1,
> |--accent1: #E8823D, --pin-active: #C9631F).',
        'Zero Layout Shift: All UI controls, modals, sheets, and floating toolbars dynamically adapt
> |to theme tokens with seamless CSS transitions.',
        'Mobile & Desktop Responsive: Optimized for phone screens, tablets, and wide desktop
> |displays with responsive split layouts.',
      ],
    },
  ];

  export function buildCodePdf() {
    const doc = new jsPDF({ unit: 'pt', format: 'a4', orientation: 'portrait' });
    const pageWidth = doc.internal.pageSize.getWidth();
    const pageHeight = doc.internal.pageSize.getHeight();
    const marginX = 36;
    const marginTop = 48;
    const marginBottom = 40;
    const contentWidth = pageWidth - marginX * 2;

    // 1. Read all files first
    const fileData = FILES_TO_INCLUDE.map((spec) => {
      const fullPath = path.resolve(PROJECT_ROOT, spec.path);
      let content = '';
      let lineCount = 0;
      try {
        content = fs.readFileSync(fullPath, 'utf8');
        lineCount = content.split('\n').length;
      } catch (e) {
        content = `// Error reading file ${spec.path}`;
      }
      return { ...spec, content, lineCount };
    });

    const totalLines = fileData.reduce((acc, f) => acc + f.lineCount, 0);

    // ========================================
    // PAGE 1: COVER PAGE
    // ========================================
    // Top Banner
    doc.setFillColor(30, 58, 138); // Navy 900
    doc.rect(0, 0, pageWidth, 16, 'F');

    // Decorative Accent bar
    doc.setFillColor(59, 130, 246); // Blue 500
    doc.rect(0, 16, pageWidth, 4, 'F');
```

scripts/generate_code_pdf.ts (continued from previous page)

```
201 |
202 |     // App Title
203 |     doc.setFont('helvetica', 'bold');
204 |     doc.setFontSize(32);
205 |     doc.setTextColor(30, 58, 138);
206 |     doc.text('Pinboard Notes', marginX, 86);
207 |
208 |     doc.setFont('helvetica', 'normal');
209 |     doc.setFontSize(14);
210 |     doc.setTextColor(71, 85, 105);
211 |     doc.text('Complete Architectural Specification & Unabridged Source Code', marginX, 110);
212 |
213 |     // Divider
214 |     doc.setDrawColor(203, 213, 225);
215 |     doc.setLineWidth(1.5);
216 |     doc.line(marginX, 126, pageWidth - marginX, 126);
217 |
218 |     // Key Metrics Badges
219 |     const badgeY = 142;
220 |     const badgeWidth = (contentWidth - 24) / 4;
221 |     const badgeHeight = 44;
222 |
223 |     const metrics = [
224 |       { label: 'SOURCE FILES', value: `${FILES_TO_INCLUDE.length} Files` },
225 |       { label: 'TOTAL CODE', value: `${totalLines.toLocaleString()} Lines` },
226 |       { label: 'FEATURE SPECS', value: '10 Core Modules' },
227 |       { label: 'RUNTIME', value: 'React 19 + Vite' },
228 |     ];
229 |
230 |     metrics.forEach((m, idx) => {
231 |       const bx = marginX + idx * (badgeWidth + 8);
232 |       doc.setFillColor(241, 245, 249);
233 |       doc.roundedRect(bx, badgeY, badgeWidth, badgeHeight, 5, 5, 'F');
234 |       doc.setFont('helvetica', 'bold');
235 |       doc.setFontSize(8);
236 |       doc.setTextColor(100, 116, 139);
237 |       doc.text(m.label, bx + 10, badgeY + 16);
238 |       doc.setFontSize(11);
239 |       doc.setTextColor(15, 23, 42);
240 |       doc.text(m.value, bx + 10, badgeY + 33);
241 |     });
242 |
243 |     // Project Overview Card
244 |     const overviewY = 202;
245 |     doc.setFillColor(248, 250, 252);
246 |     doc.roundedRect(marginX, overviewY, contentWidth, 90, 6, 6, 'F');
247 |     doc.setDrawColor(226, 232, 240);
248 |     doc.setLineWidth(1);
249 |     doc.roundedRect(marginX, overviewY, contentWidth, 90, 6, 6, 'S');
250 |
251 |     doc.setFont('helvetica', 'bold');
252 |     doc.setFontSize(11.5);
253 |     doc.setTextColor(30, 58, 138);
254 |     doc.text('Executive Summary', marginX + 16, overviewY + 22);
255 |
256 |     doc.setFont('helvetica', 'normal');
257 |     doc.setFontSize(9);
258 |     doc.setTextColor(51, 65, 85);
259 |     const summaryText =
260 |       'Pinboard Notes is a production-grade hierarchical note-taking application engineered with
    > |React 19, TypeScript, and Tailwind CSS. ' +
261 |       'This comprehensive documentation captures every feature down to the last bit: hierarchical
    > |tree persistence, chronological pinned note importance, ' +
262 |       'custom hold-and-drag note arrangement, editable detached bubble pins with transparency and
    > |viewport cropping, standout trigger controls, ' +
263 |       'smart Recycle Bin restoration to parent writing pages, full-text instant search, and dual PDF
    > |generation engines.';
264 |     const summaryLines = doc.splitTextToSize(summaryText, contentWidth - 32);
265 |     doc.text(summaryLines, marginX + 16, overviewY + 38);
```

scripts/generate_code_pdf.ts (continued from previous page)

```
266 |
267 |     // Table of Contents Preview
268 |     doc.setFont('helvetica', 'bold');
269 |     doc.setFontSize(13);
270 |     doc.setTextColor(30, 58, 138);
271 |     doc.text('Document Structure & Contents', marginX, 318);
272 |
273 |     let tocY = 340;
274 |     doc.setFontSize(9);
275 |
276 |     // Section 1 Header
277 |     doc.setFillColor(239, 246, 255);
278 |     doc.rect(marginX, tocY - 11, contentWidth, 18, 'F');
279 |     doc.setFont('helvetica', 'bold');
280 |     doc.setTextColor(29, 78, 216);
281 |     doc.text('PART I: ARCHITECTURAL SPECIFICATIONS & FEATURE MANUAL', marginX + 8, tocY + 2);
282 |     doc.text('Page 2 - 4', marginX + contentWidth - 60, tocY + 2);
283 |     tocY += 24;
284 |
285 |     FEATURE_SPECS.forEach((spec, i) => {
286 |       doc.setFont('helvetica', 'bold');
287 |       doc.setTextColor(51, 65, 85);
288 |       doc.text(`[${spec.id}]`, marginX + 12, tocY);
289 |       doc.setFont('helvetica', 'normal');
290 |       doc.text(spec.title, marginX + 44, tocY);
291 |       tocY += 15;
292 |     });
293 |
294 |     tocY += 8;
295 |
296 |     // Section 2 Header
297 |     doc.setFillColor(239, 246, 255);
298 |     doc.rect(marginX, tocY - 11, contentWidth, 18, 'F');
299 |     doc.setFont('helvetica', 'bold');
300 |     doc.setTextColor(29, 78, 216);
301 |     doc.text('PART II: UNABRIDGED SOURCE CODE (23 FILES)', marginX + 8, tocY + 2);
302 |     doc.text('Page 5+', marginX + contentWidth - 60, tocY + 2);
303 |     tocY += 22;
304 |
305 |     fileData.slice(0, 10).forEach((file, i) => {
306 |       doc.setFont('courier', 'bold');
307 |       doc.setTextColor(30, 41, 59);
308 |       doc.text(file.path, marginX + 12, tocY);
309 |       doc.setFont('helvetica', 'normal');
310 |       doc.setTextColor(100, 116, 139);
311 |       doc.text(`${file.lineCount} lines`, marginX + 220, tocY);
312 |       doc.text(file.category, marginX + 300, tocY);
313 |       tocY += 14;
314 |     });
315 |     doc.setFont('helvetica', 'italic');
316 |     doc.setTextColor(148, 163, 184);
317 |     doc.text(`... and 13 additional source and configuration files printed completely below.`,
    > |marginX + 12, tocY);
318 |
319 |     drawPageFooter(doc, marginX, contentWidth, pageHeight);
320 |
321 |     // ========================================
322 |     // PART I: FEATURE SPECIFICATIONS (Pages 2-4)
323 |     // ========================================
324 |     doc.addPage();
325 |     drawSectionBanner(doc, marginX, marginTop, contentWidth, 'PART I: ARCHITECTURAL SPECIFICATIONS &
    > |FEATURE MANUAL');
326 |
327 |     let curY = marginTop + 40;
328 |
329 |     FEATURE_SPECS.forEach((feat, idx) => {
330 |       // Check if new page needed
331 |       if (curY + 110 > pageHeight - marginBottom) {
332 |         drawPageFooter(doc, marginX, contentWidth, pageHeight);
```

scripts/generate_code_pdf.ts (continued from previous page)

```
333 |         doc.addPage();
334 |         curY = marginTop;
335 |         drawSectionBanner(doc, marginX, curY, contentWidth, 'PART I: ARCHITECTURAL SPECIFICATIONS
    > |(CONTINUED)');
336 |         curY += 40;
337 |       }
338 |
339 |       // Feature Card
340 |       doc.setFillColor(248, 250, 252);
341 |       doc.roundedRect(marginX, curY, contentWidth, 24, 4, 4, 'F');
342 |       doc.setDrawColor(226, 232, 240);
343 |       doc.setLineWidth(0.8);
344 |       doc.roundedRect(marginX, curY, contentWidth, 24, 4, 4, 'S');
345 |
346 |       doc.setFont('helvetica', 'bold');
347 |       doc.setFontSize(10.5);
348 |       doc.setTextColor(30, 58, 138);
349 |       doc.text(`${feat.id} — ${feat.title}`, marginX + 10, curY + 16);
350 |
351 |       curY += 32;
352 |
353 |       doc.setFont('helvetica', 'italic');
354 |       doc.setFontSize(8.5);
355 |       doc.setTextColor(100, 116, 139);
356 |       doc.text(feat.subtitle, marginX + 10, curY);
357 |       curY += 14;
358 |
359 |       doc.setFont('helvetica', 'normal');
360 |       doc.setFontSize(8);
361 |       doc.setTextColor(51, 65, 85);
362 |
363 |       feat.details.forEach((det) => {
364 |         const splitDet = doc.splitTextToSize(`• ${det}`, contentWidth - 20);
365 |         splitDet.forEach((line: string) => {
366 |           if (curY > pageHeight - marginBottom - 15) {
367 |             drawPageFooter(doc, marginX, contentWidth, pageHeight);
368 |             doc.addPage();
369 |             curY = marginTop;
370 |             drawSectionBanner(doc, marginX, curY, contentWidth, 'PART I: ARCHITECTURAL
    > |SPECIFICATIONS (CONTINUED)');
371 |             curY += 40;
372 |           }
373 |           doc.text(line, marginX + 12, curY);
374 |           curY += 11;
375 |         });
376 |         curY += 2;
377 |       });
378 |
379 |       curY += 10;
380 |     });
381 |
382 |     drawPageFooter(doc, marginX, contentWidth, pageHeight);
383 |
384 |     // ==========================================
385 |     // PART II: COMPLETE UNABRIDGED SOURCE CODE
386 |     // ==========================================
387 |     fileData.forEach((file) => {
388 |       doc.addPage();
389 |       let y = marginTop;
390 |
391 |       // File Header Banner
392 |       doc.setFillColor(30, 58, 138);
393 |       doc.roundedRect(marginX, y, contentWidth, 30, 4, 4, 'F');
394 |
395 |       doc.setFont('courier', 'bold');
396 |       doc.setFontSize(11);
397 |       doc.setTextColor(255, 255, 255);
398 |       doc.text(file.path, marginX + 12, y + 19);
399 |
```

scripts/generate_code_pdf.ts (continued from previous page)

```
400 |       doc.setFont('helvetica', 'normal');
401 |       doc.setFontSize(8.5);
402 |       doc.setTextColor(219, 234, 254);
403 |       doc.text(`${file.category}  |  ${file.lineCount} Lines`, marginX + contentWidth - 130, y +
    > |19);
404 |
405 |       y += 38;
406 |
407 |       // Description Box
408 |       doc.setFillColor(241, 245, 249);
409 |       doc.roundedRect(marginX, y, contentWidth, 18, 3, 3, 'F');
410 |       doc.setFont('helvetica', 'normal');
411 |       doc.setFontSize(8.5);
412 |       doc.setTextColor(71, 85, 105);
413 |       doc.text(`Role: ${file.description}`, marginX + 8, y + 12);
414 |
415 |       y += 26;
416 |
417 |       // Print all code lines
418 |       const lines = file.content.split('\n');
419 |       doc.setFont('courier', 'normal');
420 |       doc.setFontSize(7.2);
421 |       const lineHeight = 10.5;
422 |
423 |       for (let i = 0; i < lines.length; i++) {
424 |         if (y + lineHeight > pageHeight - marginBottom) {
425 |           drawPageFooter(doc, marginX, contentWidth, pageHeight);
426 |           doc.addPage();
427 |           y = marginTop;
428 |
429 |           // Continuation Header
430 |           doc.setFillColor(241, 245, 249);
431 |           doc.roundedRect(marginX, y, contentWidth, 18, 3, 3, 'F');
432 |           doc.setFont('courier', 'bold');
433 |           doc.setFontSize(8);
434 |           doc.setTextColor(71, 85, 105);
435 |           doc.text(`${file.path} (continued from previous page)`, marginX + 8, y + 12);
436 |           y += 26;
437 |         }
438 |
439 |         const rawLine = lines[i];
440 |         const lineNumStr = String(i + 1).padStart(4, ' ') + ' | ';
441 |
442 |         // Alternate row tint for readability
443 |         if (i % 2 === 0) {
444 |           doc.setFillColor(252, 253, 255);
445 |           doc.rect(marginX, y - 7.5, contentWidth, lineHeight, 'F');
446 |         }
447 |
448 |         // Line number
449 |         doc.setFont('courier', 'bold');
450 |         doc.setTextColor(148, 163, 184);
451 |         doc.text(lineNumStr, marginX + 4, y);
452 |
453 |         // Code text
454 |         doc.setFont('courier', 'normal');
455 |         const wrapped = doc.splitTextToSize(rawLine || ' ', contentWidth - 52);
456 |         renderCodeLineWithSyntaxColor(doc, wrapped[0], marginX + 44, y);
457 |
458 |         // Handle extra wide lines
459 |         if (wrapped.length > 1) {
460 |           for (let w = 1; w < wrapped.length; w++) {
461 |             y += lineHeight;
462 |             if (y + lineHeight > pageHeight - marginBottom) {
463 |               drawPageFooter(doc, marginX, contentWidth, pageHeight);
464 |               doc.addPage();
465 |               y = marginTop + 22;
466 |             }
467 |             doc.setFont('courier', 'bold');
```

scripts/generate_code_pdf.ts (continued from previous page)

```
468 |             doc.setTextColor(148, 163, 184);
469 |             doc.text('     > | ', marginX + 4, y);
470 |             doc.setFont('courier', 'normal');
471 |             doc.setTextColor(71, 85, 105);
472 |             doc.text(wrapped[w], marginX + 44, y);
473 |           }
474 |         }
475 |
476 |         y += lineHeight;
477 |       }
478 |
479 |       drawPageFooter(doc, marginX, contentWidth, pageHeight);
480 |     });
481 |
482 |     // Save PDF to public directory
483 |     fs.mkdirSync(path.dirname(OUTPUT_PATH), { recursive: true });
484 |     const pdfBytes = doc.output('arraybuffer');
485 |     fs.writeFileSync(OUTPUT_PATH, Buffer.from(pdfBytes));
486 |     const totalPages = (doc.internal as any).getNumberOfPages();
487 |     console.log(`Successfully generated full PDF to ${OUTPUT_PATH} (${totalPages} pages,
    > |${(pdfBytes.byteLength / 1024).toFixed(1)} KB)`);
488 |   }
489 |
490 |   function drawSectionBanner(doc: jsPDF, marginX: number, y: number, contentWidth: number, text:
    > |string) {
491 |     doc.setFillColor(30, 58, 138);
492 |     doc.roundedRect(marginX, y, contentWidth, 26, 4, 4, 'F');
493 |     doc.setFont('helvetica', 'bold');
494 |     doc.setFontSize(10);
495 |     doc.setTextColor(255, 255, 255);
496 |     doc.text(text, marginX + 12, y + 17);
497 |   }
498 |
499 |   function drawPageFooter(doc: jsPDF, marginX: number, contentWidth: number, pageHeight: number) {
500 |     const pageNum = (doc.internal as any).getCurrentPageInfo().pageNumber;
501 |     doc.setFont('helvetica', 'normal');
502 |     doc.setFontSize(7.5);
503 |     doc.setTextColor(148, 163, 184);
504 |     doc.text('Pinboard Notes — Complete Architectural Specification & Source Code', marginX,
    > |pageHeight - 18);
505 |     doc.text(`Page ${pageNum}`, marginX + contentWidth - 36, pageHeight - 18);
506 |   }
507 |
508 |   function renderCodeLineWithSyntaxColor(doc: jsPDF, line: string, x: number, y: number) {
509 |     const trimmed = line.trim();
510 |
511 |     // Comment
512 |     if (trimmed.startsWith('//') || trimmed.startsWith('/*') || trimmed.startsWith('*')) {
513 |       doc.setTextColor(100, 116, 139); // Slate gray
514 |       doc.text(line || ' ', x, y);
515 |       return;
516 |     }
517 |
518 |     // Import / Export statements
519 |     if (trimmed.startsWith('import ') || trimmed.startsWith('export ')) {
520 |       doc.setTextColor(14, 116, 144); // Cyan / Blue-indigo
521 |       doc.text(line || ' ', x, y);
522 |       return;
523 |     }
524 |
525 |     // Interface / Type declarations
526 |     if (trimmed.startsWith('interface ') || trimmed.startsWith('type ') || trimmed.startsWith('const
    > |') || trimmed.startsWith('function ')) {
527 |       doc.setTextColor(67, 56, 202); // Indigo
528 |       doc.text(line || ' ', x, y);
529 |       return;
530 |     }
531 |
532 |     // Standard code line
```

scripts/generate_code_pdf.ts (continued from previous page)

```
533 |     doc.setTextColor(30, 41, 59); // Slate 800
534 |     doc.text(line || ' ', x, y);
535 |   }
536 |
537 |   // Execute generator
538 |   buildCodePdf();
539 |
```